Artificial Intelligence 358 (2026) 104586

Contents lists available at ScienceDirect

Artificial Intelligence

journal homepage: www.elsevier.com/locate/artint

Robust and effective multi-agent path execution with timing uncertainty

Yihao Liu [a], Xueyan Tang [a,*], Wentong Cai [a], Jingning Li [b]

[a] College of Computing and Data Science, Nanyang Technological University, Singapore
[b] NCS Pte Ltd, Singapore

ARTICLE INFO

Keywords:
Multi-agent plan execution
Timing uncertainty
Collision avoidance
Deadlock avoidance
Coordination

ABSTRACT

In many multi-agent applications, agents such as robots and automated guided vehicles move around concurrently in a shared workspace to carry out tasks. The path plans of agents are typically computed beforehand to avoid collisions and deadlocks. However, in real-world systems, unexpected conditions during plan execution can break the assumptions made in path planning and disturb path execution. In this paper, we develop a robust and effective execution framework of multi-agent path plans to deal with unexpected delays arising from unforeseen circumstances. The framework tracks the evolving locations of agents and dynamically chooses a maximal set of agents to move together while guaranteeing conflict-freeness and deadlock-freeness. To ensure that agents can always reach their destinations, we formulate a feasibility problem to check whether all agents can successfully execute the rest of their path plan when choosing which agents to move. We prove that this problem is NP-complete and design feasibility test algorithms. Leveraging feasibility tests, we further develop algorithms to coordinate agent movements in an online fashion. Several optimizations are also implemented to improve the efficiency and scalability of the feasibility test and coordination algorithms. We experimentally demonstrate the execution effectiveness and computational efficiency of our algorithms.

1. Introduction

The planning and execution of multi-agent systems is a fundamental problem with broad applications in areas such as warehouse automation, autonomous vehicle coordination, and multi-robot exploration. In these applications, agents such as robots and automated guided vehicles move around concurrently in a shared workspace to carry out tasks. Efficiently coordinating multiple agents to move without conflicts is crucial for improving safety, scalability, and operational efficiency in these systems. Path planning and plan execution are two essential components to address this problem. Path planning constructs collision-free paths for agents to move from their current locations to their assigned destinations. Plan execution, in contrast, focuses on the actual implementation of these plans in dynamic environments. Robust and effective execution is not only critical but also challenging because real-world multi-agent systems often experience unexpected conditions due to internal factors (e.g., robot kinematic constraints, slips and hardware failures) and external factors (e.g., environment disturbances, communication latency, clock drifts and human-in-the-loop operations) where agents may fail to move successfully on time [1,2]. For example, when a robot breaks down, it may stay at its current location indefinitely; when there are humans ahead, an autonomous vehicle may have to stop to prevent colliding with them; in many automated

* Corresponding author.
E-mail addresses: yihao002@e.ntu.edu.sg (Y. Liu), asxytang@ntu.edu.sg (X. Tang), aswtcai@ntu.edu.sg (W. Cai), jingning.li@ncs.com.sg (J. Li).

https://doi.org/10.1016/j.artint.2026.104586
Received 15 December 2024; Received in revised form 10 June 2026; Accepted 11 June 2026
Available online 15 June 2026

warehouses, robots transport packages packed by humans. As a result, plan execution is commonly accompanied by timing uncertainty in the form unexpected delays in agent movements due to unforeseen circumstances. Such delays can break the assumptions made in path planning, threatening the safety and degrading the effectiveness of plan execution. If not handled properly, they can adversely affect and slow down the plan execution of many agents in a cascading manner.

The design of execution policies typically involves a trade-off between execution effectiveness and computational effort. On the one hand, computational overhead can be minimized by keeping to the original plan despite unexpected delays, but this approach can result in unnecessary waiting of agents during execution. On the other hand, execution effectiveness can be maximized by replanning the paths for agents whenever unexpected delays arise, but this approach can give rise to high computational cost. In this paper, we develop a new execution framework of multi-agent path plans to deal with timing uncertainty. Our framework operates at a strategic point on the Pareto frontier of the aforesaid trade-off. To save computational effort, instead of replanning the paths for agents, we simply adjust the order for the agents to occupy forthcoming common nodes along their paths dynamically without compromising the feasibility of the remaining execution. To do so, we introduce a spatio-temporal dependency representation called the Location Dependency Graph (LDG) and formulate a feasibility problem to determine whether the remaining portion of a path plan can be successfully executed without conflicts and deadlocks. We prove that the problem is NP-complete and design sound and efficient feasibility test algorithms. Making use of feasibility tests, we then develop algorithms to coordinate agent movements in an online fashion and enable as many agents as possible to move concurrently to maximize the effectiveness of execution while guaranteeing conflict-freeness and deadlock-freeness. Several optimizations are also proposed to improve the efficiency and scalability of the feasibility test and coordination algorithms. Furthermore, we conduct experiments to demonstrate the execution effectiveness and computational efficiency of our algorithms.

A preliminary version of this work was presented at SoCS 2024 [3]. This paper develops new techniques for feasibility tests, expands the discussions of the proposed framework, and conducts extended empirical evaluations. The rest of the paper is organized as follows: Section 2 summarizes the related work and contrasts our work with the literature. Section 3 introduces a formal problem setup of our work. Section 4 studies the feasibility of path plans and proposes feasibility test algorithms. Section 5 develops algorithms to coordinate agent movements online. Section 6 presents empirical evaluations of the proposed framework. Finally, Section 7 concludes the paper.

2. Related work

In general, there are two strategies to deal with timing uncertainty that may arise during plan execution. One strategy is to cater for uncertainty in offline path planning before execution by assuming maximum delays or probabilities of delays. The other strategy is to carefully control plan execution online to guarantee safe execution and repair or reschedule the plan when needed. Our solution follows the latter strategy. In this section, we review the related work on these two strategies.

2.1. Catering for uncertainty in path planning

Path planning is often known as Multi-Agent Path Finding (MAPF), which seeks a cost-optimized, conflict-free plan for a group of moving agents in the shared workspace [4]. Classical MAPF discretizes time into unit timesteps and assumes that an agent either waits at its current location or moves to an adjacent location in each timestep. A number of recent MAPF formulations attempt to cater for uncertainty in agent movements. The k-robust MAPF problem [5] allows each agent to be delayed for up to k timesteps. The MAPF with Time Uncertainty problem [6] assumes upper and lower bounds on the timesteps taken to move between adjacent locations. The limitations of these works are that the plan may become invalid when the assumed delay bounds are violated and may be far from optimal in the cost objective if the bounds are loose. The MAPF with Uncertainty problem [7] and the p-robust MAPF problem [8] pursue plans that can be executed without conflicts with probabilistic guarantees. The MAPF with Delay Probabilities problem [9] assumes the same knowledge and aims to find a plan to minimize the expected makespan (the earliest time when all agents have reached their destinations). These methods rely on the knowledge of the probability that an agent will be delayed when moving between adjacent locations, which is often hard to obtain in practice. Different from the above studies, we develop efficient heuristics to keep plan execution effective online without assuming any prior knowledge on the delays that may arise in the execution.

A recent work formulated an offline time-independent path planning problem [10], which seeks a set of paths guaranteeing that all agents will reach their destinations, regardless of how they are scheduled, under all timing uncertainties. It derived a necessary and sufficient condition for such agent paths to exist and proved that finding such a solution is NP-hard and verifying a solution candidate is co-NP-complete. By contrast, our framework does not require that the input agent paths can be executed with any schedule under all timing uncertainties. We just assume that they are feasible to execute without timing uncertainty and aim to always keep the remaining agent paths executable to completion at runtime as timing uncertainties arise. Our hardness result for determining the feasibility of a path plan is orthogonal to the theoretical establishments in [10].

2.2. Catering for uncertainty in plan execution

One common approach of this strategy is to construct a directed graph from the MAPF solution to model precedence dependencies among events or actions for plan execution to follow. Hönig et al. [11] defined the Temporal Plan Graph (TPG) to capture the order in which the agents pass through every location in a MAPF solution. Ma et al. [9] designed a plan execution policy called Minimal Communication Policy (MCP) to avoid conflicts by requiring agents to adhere to the dependencies specified in the TPG. Hönig et

al. [12] extended the TPG to the Action Dependency Graph (ADG) by focusing on the precedence relation between actions rather than the events of agents entering locations. Sticking to a static TPG/ADG, however, can lead to ineffective execution when facing unexpected delays as a delay on one agent may result in unnecessary waits of other agents.

Several follow-up studies [13–16] optimized the TPG/ADG by switching precedence dependencies during plan execution via mixed-integer programming or A* search, etc. These approaches generally have high computational overhead because they strive for a new plan optimizing the cost assuming no further delay, which nevertheless may be unnecessary if further delays may occur in the future. In contrast, we focus on which agents can move together immediately given their current locations rather than a full solution to execute the remaining path plan to completion. In our solution, we introduce a graph representation called Location Dependency Graph (LDG) to capture all potential dependencies among agents and enable using alternative dependencies upon unexpected delays.

Some recent work considered problem settings close to our work. Švancara et al. [17] studied finding a MAPF solution given a predefined path for each agent with the possibility of adding wait actions. They presented an extended wait-graph (similar to TPG) to order the occupation of every node along the paths of all agents and established the equivalence between the existence of an acyclic wait-graph and a valid MAPF solution (similar to our Theorem 1 in Section 4.1). However, they did not develop any algorithm to decide whether an acyclic wait-graph exists.

Concurrent to the preliminary version of our work [13], Kottinger et al. [18] addressed repairing a MAPF solution after encountering unexpected delays by introducing additional delays to some agents so that the remainder of the plan can be executed safely. It is competent in coping with a single unexpected delay of an agent. An natural extension of [18]'s approach to deal with succeeding unexpected delays is to repair the MAPF solution iteratively upon every unexpected delay. This however may not be effective because additional delays introduced in all repairs will have to be aggregated since [18]'s approach only allows adding delays, not removing delays. Conceptually, our approach can be considered as rescheduling the remaining path plan when a new unexpected delay arises, where it is possible to overwrite the previous schedule. This can enhance the effectiveness of plan execution when unexpected delays arise progressively over time.

Su et al. [19] introduced the Bidirectional Temporal Plan Graph (BTPG) to facilitate fast switching of precedence dependencies during plan execution. However, it requires all switchable dependencies to be compatible with each other (the plan must be kept valid by all their possible combinations), which limits the dependencies that can be switched. Our approach enables much wider dependency switching options than BTPG. We shall contrast and compare our solution with BTPG in the experiment section.

We also note that Zhang et al. [20] developed a concurrent planning and execution framework in MAPF, which iteratively commits a window of k steps of the plan for execution. During the commit window, the planner can further compute or optimize the remaining uncommitted plan. After k timesteps of execution, the next k steps of the plan will be committed. Recently, the same authors [21] further extended the framework to handle unexpected delays during execution by running dummy simulation at the end of a commit window to cater for committed but not executed actions and decide the next k actions for the subsequent commit window. The above framework can be instantiated with existing execution policies including MCP [9] and PIBT [22]. Our work complements [20,21] in that we seek to develop a more effective execution policy. Both MCP and PIBT policies are used as baselines in our experimental evaluation.

3. Problem statement

Consider an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where the nodes in $\mathcal{V}$ correspond to the locations in which agents can stay and the edges in $\mathcal{E}$ corresponds to the connections between nodes along which agents can move. There is a set of M agents $\{a_1, a_2, \dots, a_M\}$. Each agent a_i has a start location $s_i \in \mathcal{V}$, a goal location $g_i \in \mathcal{V}$, and a predefined path p_i from s_i to g_i expressed by a sequence of nodes $v_{i,1} \to v_{i,2} \to \cdots \to v_{i,n_i}$ where each $\{v_{i,j}, v_{i,j+1}\} \in \mathcal{E}$, $v_{i,1} = s_i$, $v_{i,n_i} = g_i$ and n_i is the length of the path. To execute a path plan $\mathcal{P} = \{p_1, p_2, \dots, p_M\}$, each agent a_i moves from its start location s_i to its goal location g_i through the path p_i. In general, the path of an agent is not required to be cycle-free. If the path of an agent contains a cycle, the same node is repeated in the sequence. With node repetitions allowed, a path is usually called a walk in graph theory. We note that the output of a MAPF solver typically provides a timestamped path for each agent, which specifies particular locations of the agent at particular times. Our representation of a path plan is more general than a MAPF solution in that we do not associate timestamps with locations or actions. It can model any agent paths specified by applications such as security patrolling.

Assume time is discretized. In each timestep, an agent executes either a *wait* action or a *move* action. In a *wait* action, the agent stays at its current node until the next timestep. In a *move* action, the agent goes towards an adjacent node, but whether it can arrive at the adjacent node successfully before the next timestep is not guaranteed, which results in the *timing uncertainty* that we aim to deal with. If the agent fails to reach the adjacent node, it must continue executing the *move* action in the next timestep, i.e., the *move* action cannot be canceled before the agent arrives at the adjacent node.

We consider two classical types of conflicts: node conflicts where two agents stay at the same node at the same timestep; and edge conflicts where two agents move along the same edge in opposite directions at the same timestep. In addition, we also consider following conflicts where an agent goes towards a node occupied by another agent in the previous timestep [4], because owing to unexpected delays, if the latter agent fails to move and the former agent succeeds in moving, they may collide at the node. To avoid conflicts, each agent is conceptually considered to occupy some node(s) at any time in path execution. Specifically, when an agent executes a *wait* action, it occupies the current node where it stays. When an agent executes a *move* action, we consider it occupying both the current node and the adjacent node it is moving to, until the agent arrives at the adjacent node. In path execution, we impose the constraint that any two agents cannot occupy the same node concurrently. Such a constraint addresses all the aforesaid

node, edge and following conflicts. We assume no violation of the constraint in the initial (resp. final) state, i.e., the start (resp. goal) locations of all agents are distinct. It is, however, possible for a start location s_i to be the same as a goal location g_j.

Given a path plan $\mathcal{P}$, we would like to coordinate the *move* and *wait* actions taken by agents during the execution to ensure that all agents eventually arrive at their goal locations. Since we do not assume any prior knowledge on the delays that may arise in the execution, it is difficult to define an objective function directly. Instead, we use the number of agents taking *move* actions as a measure of path execution effectiveness and aim to maximize this number at any time, which intuitively would help optimize classic cost objectives such as makespan or sum-of-costs.

We develop a set of algorithmic solutions to coordinate the path execution online. Our main approach is to track the evolving locations of agents and dynamically choose a maximal set of agents to move with conflict-freeness and deadlock-freeness guarantees. We start by creating a graph representation called Location Dependency Graph (LDG) to model dependencies among agents in the execution to avoid conflicts and deadlocks (Section 4.1). To ensure that agents can always reach their goal locations following their predefined paths, we define a feasibility problem to determine whether the remaining portion of a path plan can be successfully executed. We prove that the problem is NP-complete (Section 4.2) and design sound and efficient feasibility test algorithms (Section 4.3). The feasibility tests are then used as building blocks to construct algorithms for choosing which agents to move in the execution with the objective of enabling as many agents as possible to move concurrently (Section 5).

4. Path plan feasibility

4.1. Feasibility and location dependency graph

Not all path plans can be successfully executed. A path plan is said to be *feasible* if all agents can arrive at their goal locations following the paths while meeting the constraint that agents never occupy the same node simultaneously; otherwise, the path plan is *infeasible*.

If the paths in a path plan $\mathcal{P}$ do not share any common nodes, the plan is guaranteed to be feasible, as each agent can continuously perform *move* actions without any potential conflicts. If there exist common nodes among different paths, to execute the path plan, we need to decide the order for the agents to occupy common nodes, which can be modeled by a location dependency graph.

We refer to each node $v_{i,j}$ on an agent a_i's path p_i as a *location state*. The location dependency graph (LDG) is a directed graph $\mathcal{G}_{\text{LDG}} = (\mathcal{V}_{\text{LDG}}, \mathcal{E}_{\text{LDG}})$, where $\mathcal{V}_{\text{LDG}} = \{v_{i,j} \mid 1 \leqslant i \leqslant M, 1 \leqslant j \leqslant n_i\}$ is the set of all possible location states of the agents, and $\mathcal{E}_{\text{LDG}}$ specifies the precedence constraints for the agents to achieve their location states. $\mathcal{E}_{\text{LDG}}$ consists of four parts.

The first part of $\mathcal{E}_{\text{LDG}}$ is based on the path of each individual agent and includes the edges $\{\langle v_{i,j}, v_{i,j+1}\rangle \mid 1 \leqslant i \leqslant M, 1 \leqslant j < n_i\}$, meaning that agent a_i cannot achieve the location state $v_{i,j+1}$ before $v_{i,j}$.

The second part of $\mathcal{E}_{\text{LDG}}$ is based on the initial location states of the agents: for each agent a_i, if its initial location state $v_{i,1}$ shares the same node in the graph $\mathcal{G}$ with a non-initial location state $v_{i',j}$ of another agent $a_{i'}$, we add an edge $\langle v_{i,2}, v_{i',j}\rangle$ because $a_{i'}$ cannot achieve the location state $v_{i',j}$ before a_i moves away from its start location.[1]

The third part of $\mathcal{E}_{\text{LDG}}$ is based on the final location states of the agents: for each agent a_i, if its final location state v_{i,n_i} shares the same node in the graph $\mathcal{G}$ with a non-final location state $v_{i',j}$ of another agent $a_{i'}$, we add an edge $\langle v_{i',j+1}, v_{i,n_i}\rangle$ because $a_{i'}$ must pass the location state $v_{i',j}$ before a_i moves to its goal location. Otherwise, once a_i moves to its goal location and stays there infinitely, it will be impossible for $a_{i'}$ to achieve the location state $v_{i',j}$.

The above three parts of $\mathcal{E}_{\text{LDG}}$ are called *predetermined edges* because these precedence constraints are fixed in any path execution.

The last part of $\mathcal{E}_{\text{LDG}}$ is to resolve the order of two non-initial and non-final location states from different agents that share the same node in the graph $\mathcal{G}$. Recall that any two agents cannot occupy the same node simultaneously. Thus, for each pair of location states $v_{i,j}$ and $v_{i',j'}$ from two respective agents a_i and $a_{i'}$ sharing a common node, an edge $\langle v_{i,j+1}, v_{i',j'}\rangle$ can be formed if a_i achieves $v_{i,j}$ before $a_{i'}$ achieves $v_{i',j'}$, or an edge $\langle v_{i',j'+1}, v_{i,j}\rangle$ can be formed if $a_{i'}$ achieves $v_{i',j'}$ before a_i achieves $v_{i,j}$. We refer to these two edges as a pair of *unsettled edges*. To ensure conflict-free path execution, one of them must be chosen and added to the LDG. Note that by definition, different pairs of location states must have distinct unsettled edges. We refer to the LDG with predetermined edges only as a *partial* LDG, and refer to the LDG after choosing one edge from each pair of unsettled edges as a *complete* LDG. The following property is similar to what was stated in [13,14].

Theorem 1. *A path plan is feasible if and only if there exists an acyclic complete LDG.*

Proof. If a complete LDG is acyclic, we can derive a topological ordering of all location states in the LDG. Note that the transition from one location state to the next location state of the same agent represents the execution of a *move* action by the agent. Thus, we can let the agents take *move* actions one at a time to achieve their location states following the topological ordering, and then all agents can arrive at their goal locations without conflicts or deadlocks. Hence, the path plan is feasible.

Conversely, if the path plan is feasible, there exists an execution that allows all agents to reach their goal locations and satisfies the constraint that agents never occupy the same node simultaneously. Thus, for each common node between the paths of two agents, the agents must achieve the corresponding location states sequentially in the execution. This implies that the execution must comply with one of the precedence constraints indicated by each pair of unsettled edges. By choosing all the unsettled edges that the execution complies with, we can construct a complete LDG that is acyclic because the execution is deadlock-free. □

[1] We assume that the path of a_i includes at least two location states. If a_i's path has only one location state, it has the same start and goal locations and does not move. It is impossible for $a_{i'}$ to pass through a_i's location and the path plan is definitely infeasible.

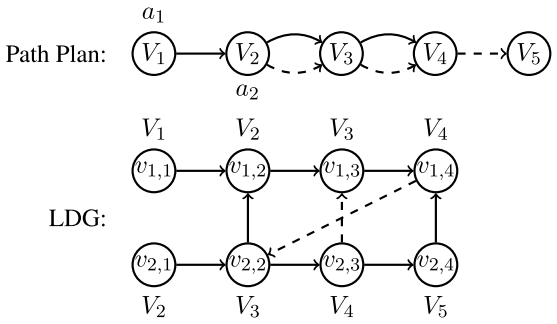


Fig. 1. A feasible path plan and its LDG.

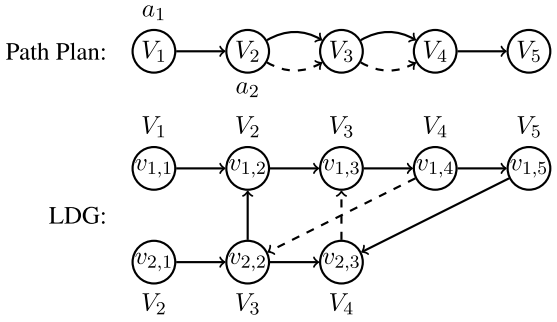


Fig. 2. An infeasible path plan and its LDG.

Fig. 1 shows a path plan for two agents, where a_1's path is marked in solid lines and a_2's path is marked in dashed lines. In the LDG, predetermined edges are illustrated by solid lines, and unsettled edges are illustrated by dashed lines. As can be seen, there is only one pair of unsettled edges for the common node V_3 between the two paths. If the edge $\langle v_{2,3}, v_{1,3}\rangle$ is chosen, there is no cycle in the complete LDG. Thus, the path plan is feasible. To execute the path plan, the complete LDG indicates that a_1 should follow a_2 and be at least one node away from a_2.

Fig. 2 shows another path plan. In the LDG, if the edge $\langle v_{2,3}, v_{1,3}\rangle$ is chosen, a cycle $v_{2,3} \to v_{1,3} \to v_{1,4} \to v_{1,5} \to v_{2,3}$ is formed. If the edge $\langle v_{1,4}, v_{2,2}\rangle$ is chosen, a cycle $v_{1,4} \to v_{2,2} \to v_{1,2} \to v_{1,3} \to v_{1,4}$ is formed. Since the complete LDG always has a cycle, the path plan is infeasible.

Our LDG differs from existing graph representations in several aspects. First, the existing Temporal Plan Graph (TPG) [9,11] and Action Dependency Graph (ADG) [12] have fixed edges (precedences) only and the execution must follow such orders. In contrast, our LDG includes unsettled edges which offer the flexibility to dynamically adjust the order for agents to occupy locations during the execution when facing unexpected delays. Second, ADG models only actions (each node in the ADG represents an action), which is inadequate because with uncertain delays, agents may need to stop at locations in the graph $\mathcal{G}$ to avoid conflicts and deadlocks even if the initial MAPF solution is to perform move actions along adjacent edges in $\mathcal{G}$ consecutively.

We also remark that our LDG is conceptually similar to the alternative graph introduced for job-shop scheduling with blocking constraints [23]. In job-shop scheduling, different jobs may have operations that must be processed on the same machine, but each machine can process only one operation at a time. In the alternative graph, fixed arcs (similar to our predetermined edges) are created to model the specified order of operations in each job, and alternative arcs (similar to our unsettled edges) are created to model possible precedence relations between operations of different jobs sharing the same machine. However, a job initially does not occupy any machine and after a job finishes, it releases the machine of the last operation. Thus, the alternative graph does not include arcs corresponding to the second and third parts of $\mathcal{E}_{\text{LDG}}$ in our LDG.

4.2. Hardness of path plan feasibility

Determining the feasibility of a path plan (named as the path plan feasibility problem) is computationally difficult.

Theorem 2. *The path plan feasibility problem is NP-complete.*

Obviously, the path plan feasibility problem is in NP. We reduce the *LSAT* (linear *SAT*) problem to the path plan feasibility problem to prove Theorem 2. The classical *SAT* (Boolean satisfiability) problem is to determine whether a formula can be made *true* by assigning appropriate logical values (*true*, *false*) to its variables. If there are at most three variables in each clause, it is called a *3-SAT* problem, which is known to be NP-complete [24]. The *LSAT* problem is a subclass of the *3-SAT* problem, where each clause intersects at most one other clause. Moreover, if two clauses intersect, they have exactly one literal in common. Recently, it has been shown that the *LSAT* problem is also NP-complete [25].

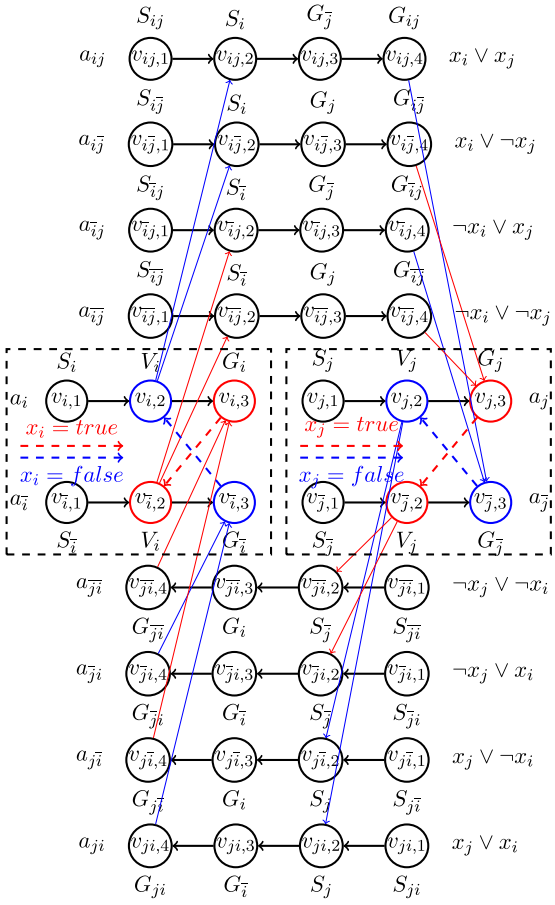


Fig. 3. LDG of variables x_i and x_j.

We reduce the *LSAT* problem to the path plan feasibility problem by constructing a path plan $\mathcal{P}$ from the variables and clauses of the *LSAT* formula, and show that determining whether it is possible to transform the partial LDG of $\mathcal{P}$ into an acyclic complete LDG is equivalent to evaluating the satisfiability of the *LSAT* formula.

For each variable x_i in the *LSAT* formula, we construct two *primary* agents a_i and $a_{\bar{i}}$, with paths $S_i \to V_i \to G_i$ and $S_{\bar{i}} \to V_i \to G_{\bar{i}}$ respectively. The location states of a_i and $a_{\bar{i}}$ are shown in the left dashed frame in the middle of Fig. 3, where predetermined edges are marked by solid lines and unsettled edges are marked by dashed lines. Due to the common node V_i, there is a pair of unsettled edges $\langle v_{i,3}, v_{\bar{i},2}\rangle$ and $\langle v_{\bar{i},3}, v_{i,2}\rangle$. The choice between these two unsettled edges will be mapped to the assignment of the variable x_i's value. If the edge $\langle v_{i,3}, v_{\bar{i},2}\rangle$ is chosen, x_i is assigned *true*; if the edge $\langle v_{\bar{i},3}, v_{i,2}\rangle$ is chosen, x_i is assigned *false*.

For each pair of variables x_i and x_j in the *LSAT* problem, we construct eight possible *auxiliary* agents a_{ij}, $a_{i\bar{j}}$, $a_{\bar{i}j}$, $a_{\bar{i}\bar{j}}$, a_{ji}, $a_{j\bar{i}}$, $a_{\bar{j}i}$, $a_{\bar{j}\bar{i}}$, and each agent is assigned a path of four nodes. The location states of these agents are shown in the upper and lower parts of Fig. 3. The start and goal locations of the agents are all different. The pair of the second and third nodes in each path is one of the four combinations of a start location from $a_i/a_{\bar{i}}$ and a goal location from $a_j/a_{\bar{j}}$ or the four combinations of a start location from $a_j/a_{\bar{j}}$ and a goal location from $a_i/a_{\bar{i}}$. Since the paths of auxiliary agents include only the start and goal locations of primary agents, only predetermined edges (solid lines) will be added to the LDG as shown in Fig. 3.

Each auxiliary agent represents a possible clause of variables x_i and x_j. For example, the agent a_{ij} represents the clause $x_i \vee x_j$, since its path includes the start location S_i of a_i and the goal location $G_{\bar{j}}$ of $a_{\bar{j}}$. Note that a_{ij}'s path is created in this way because the clause $x_i \vee x_j$ evaluates to *false* only if both x_i and x_j are assigned *false*. Based on the mapping described above, assigning *false* to x_i and x_j implies that the unsettled edges $\langle v_{\bar{i},3}, v_{i,2}\rangle$ and $\langle v_{\bar{j},3}, v_{j,2}\rangle$ are chosen. Thus, a_{ij}'s path shares common nodes with location states $v_{i,1}$ and $v_{\bar{j},3}$, so that the predetermined edges form a path $v_{i,2} \to v_{ij,2} \to v_{ij,3} \to v_{ij,4} \to v_{\bar{j},3}$ (bold solid lines in Fig. 3) to link the aforesaid unsettled edges in the LDG. Similarly, the agent a_{ji} represents the clause $x_j \vee x_i$ (which is equivalent to $x_i \vee x_j$). Hence, a_{ji}'s path shares common nodes with location states $v_{j,1}$ and $v_{\bar{i},3}$, so that the predetermined edges form another path $v_{j,2} \to v_{ji,2} \to v_{ji,3} \to v_{ji,4} \to v_{\bar{i},3}$ (bold solid lines in Fig. 3) to link the aforesaid unsettled edges in the reverse direction.

If there are m boolean variables in the *LSAT* problem, $2m$ agents are constructed to represent the variables, and at most $4m(m-1)$ agents are constructed to represent the clauses. Thus, the path plan is constructed in polynomial time.

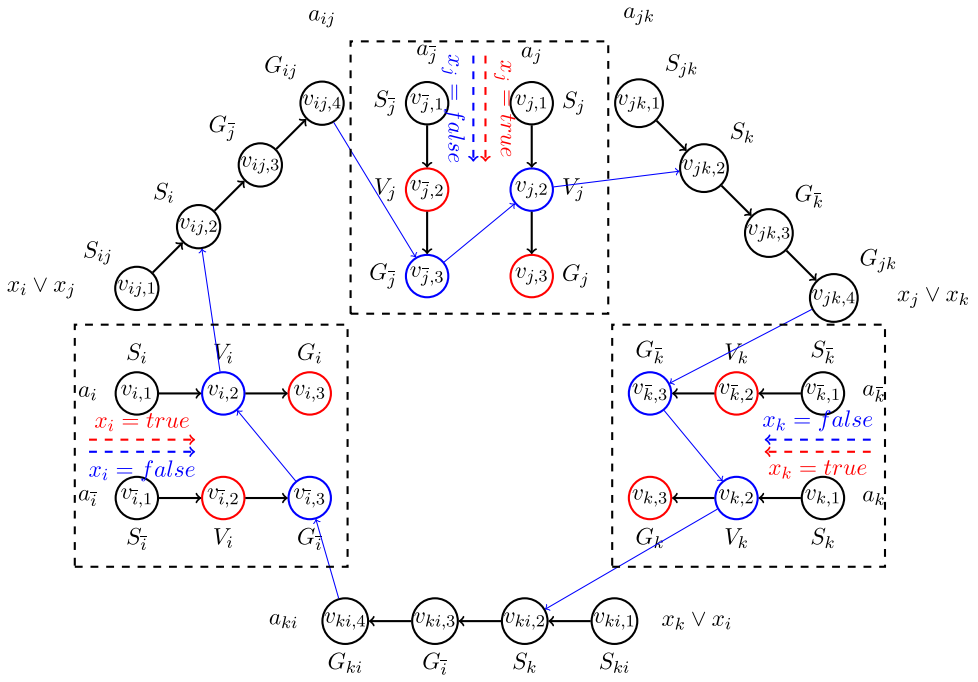


Fig. 4. A cycle formed in the LDG when a 3-literal clause $x_i \vee x_j \vee x_k$ evaluates to *false*.

Given an *LSAT* formula, the two primary agents a_i and $a_{\bar{i}}$ for each variable x_i are always included in constructing the path plan. The auxiliary agents for each pair of variables are *selectively* added to the path plan according to the clauses in the *LSAT* formula. In an *LSAT* formula, the clauses can be 2-literal or 3-literal. We consider them separately. For a 2-literal clause involving variables x_i and x_j, we include the two auxiliary agents corresponding to the literals in the clause. For example, if the clause is $x_i \vee x_j$, we include the agents a_{ij} and a_{ji}. If both x_i and x_j are assigned *false*, the clause $x_i \vee x_j$ evaluates to *false* and hence the *LSAT* formula also evaluates to *false*. Accordingly, in the path plan feasibility problem, if the unsettled edges $\langle v_{\bar{i},3}, v_{i,2}\rangle$ and $\langle v_{\bar{j},3}, v_{j,2}\rangle$ are both chosen, a cycle $(v_{\bar{i},3} \to v_{i,2} \to v_{ij,2} \to v_{ij,3} \to v_{ij,4} \to v_{\bar{j},3} \to v_{j,2} \to v_{ji,2} \to v_{ji,3} \to v_{ji,4} \to v_{\bar{i},3})$ is formed in the LDG, as can be seen from Fig. 3.

For a 3-literal clause involving variables x_i, x_j and x_k ($i < j < k$), we include the three auxiliary agents corresponding to the literals in the clause. For example, if the clause is $x_i \vee x_j \vee x_k$, we include the agents a_{ij}, a_{jk} and a_{ki}. Similarly, if all of x_i, x_j and x_k are assigned *false*, the clause $x_i \vee x_j \vee x_k$ evaluates to *false* and hence the *LSAT* formula also evaluates to *false*. Accordingly, in the path plan feasibility problem, if the unsettled edges $\langle v_{\bar{i},3}, v_{i,2}\rangle$, $\langle v_{\bar{j},3}, v_{j,2}\rangle$ and $\langle v_{\bar{k},3}, v_{k,2}\rangle$ are all chosen, a cycle $(v_{\bar{i},3} \to v_{i,2} \to v_{ij,2} \to v_{ij,3} \to v_{ij,4} \to v_{\bar{j},3} \to v_{j,2} \to v_{jk,2} \to v_{jk,3} \to v_{jk,4} \to v_{\bar{k},3} \to v_{k,2} \to v_{ki,2} \to v_{ki,3} \to v_{ki,4} \to v_{\bar{i},3})$ is formed in the LDG, as illustrated in Fig. 4.

What remains is to establish the equivalence between a satisfying assignment of the *LSAT* formula and an acyclic complete LDG of the path plan, or alternatively, the equivalence between an unsatisfying assignment and a cyclic complete LDG. First, the following property is quite obvious by the above construction.

Lemma 1. *If a variable assignment makes the LSAT formula evaluate to false, choosing the unsettled edges according to the assignments would give rise to a cycle in the complete LDG.*

Proof. If the LSAT formula evaluates to *false*, at least one clause evaluates to *false*. For each 2-literal or 3-literal clause in the LSAT formula, by the construction of the path plan (and the resulting LDG), if the clause evaluates to false, a cycle is formed in the complete LDG. Thus, if any clause evaluates to *false*, there must exist a cycle in the corresponding complete LDG. Hence, the lemma is proven. □

Next, we prove that the reverse of the above property also holds based on the characteristics of the *LSAT* formula.

Lemma 2. *If the choices of the unsettled edges lead to a cycle in the complete LDG, assigning the values to the variables according to the edge choices would make the LSAT formula evaluate to false.*

Proof. We examine the characteristics of a cycle in the complete LDG. By the construction of the path plan, from the path of any auxiliary agent, we can only go to the goal location state $v_{i,3}$ (or $v_{\bar{i},3}$) of a primary agent. If the unsettled edge incident to $v_{i,3}$ (or $v_{\bar{i},3}$) is not chosen in the complete LDG, we cannot go to any other location state and thus a cycle cannot be formed. If the unsettled edge incident to $v_{i,3}$ (or $v_{\bar{i},3}$) is chosen, we can go to the middle location state $v_{\bar{i},2}$ (or $v_{i,2}$) of the other primary agent for the same variable in *LSAT*. From $v_{\bar{i},2}$ (or $v_{i,2}$), if we go to the goal location state $v_{\bar{i},3}$ (or $v_{i,3}$) of the same agent, we cannot further go to any other location state because the unsettled edge incident to $v_{\bar{i},3}$ (or $v_{i,3}$) must not be chosen (since only one edge from each pair of unsettled edges is chosen to form a complete LDG). Thus, to form a cycle, from $v_{\bar{i},2}$ (or $v_{i,2}$), we can only go to the path of an auxiliary agent. Hence, if

a cycle is formed in a complete LDG, the cycle must involve the paths of the agents sequentially in the pattern of (an auxiliary agent, two primary agents for the same variable, an auxiliary agent, two primary agents for the same variable, …).

Recall that each auxiliary agent involved corresponds to a 2-literal clause. By the construction of the path plan, either (i) the 2-literal clause appears as a standalone clause in the *LSAT* formula, or (ii) together with the 2-literal clause of an adjacent auxiliary agent involved in the cycle, they form a 3-literal clause in the *LSAT* formula. We refer to this standalone 2-literal clause or this 3-literal clause as an *LSAT* clause involved. Note that an auxiliary agent cannot be mapped to two or more *LSAT* clauses (otherwise, these *LSAT* clauses have two literals in common, which contradicts the definition of the *LSAT* problem). Thus, each auxiliary agent involved in the cycle is mapped to exactly one *LSAT* clause.

By the aforesaid pattern of the cycle, the 2-literal clauses of two adjacent auxiliary agents involved must share a literal (of the variable represented by the two primary agents in between). This implies that if the *LSAT* clauses of two adjacent auxiliary agents are different, they must share a literal. As a result, if there are at least two *LSAT* clauses involved, every clause must share a literal with another clause at each end. If there are exactly two *LSAT* clauses involved, they must share two literals (such as $x_1 \vee x_2 \vee x_3$ and $x_3 \vee x_4 \vee x_1$), which contradicts the definition of the *LSAT* problem because two intersecting clauses must have exactly one literal in common. If there are three or more *LSAT* clauses involved, each of them must intersect two other clauses (such as $x_1 \vee x_2 \vee x_3$, $x_3 \vee x_4 \vee x_5$ and $x_5 \vee x_6 \vee x_1$), which also contradicts the definition of the *LSAT* problem because each clause can intersect at most one other clause. Therefore, there can only be one *LSAT* clause involved.

If the *LSAT* clause involved is a 2-literal clause (such as $x_i \vee x_j$), the cycle involves two auxiliary agents (such as a_{ij}, a_{ji}) and the primary agents for the two variables in the 2-literal clause (such as a_i, $a_{\bar{i}}$, a_j, $a_{\bar{j}}$). By the variable assignment corresponding to the unsettled edges chosen in the complete LDG, the 2-literal clause must evaluate to *false*.

Similarly, if the *LSAT* clause involved is a 3-literal clause (such as $x_i \vee x_j \vee x_k$), the cycle involves three auxiliary agents (such as a_{ij}, a_{jk}, a_{ki}) and the primary agents for the three variables in the 3-literal clause (such as a_i, $a_{\bar{i}}$, a_j, $a_{\bar{j}}$, a_k, $a_{\bar{k}}$). By the variable assignment corresponding to the unsettled edges chosen in the complete LDG, the 3-literal clause must evaluate to *false*.

Finally, if one clause evaluates to *false*, the *LSAT* formula evaluates to *false*. Hence, the lemma is proven. □

Note that there is a one-to-one correspondence between the variable assignments and the complete LDGs. In addition, an *LSAT* formula is satisfiable if and only if there exists a satisfying assignment. By Theorem 1, a path plan is feasible if and only if there exists an acyclic complete LDG. Therefore, by Lemmas 1 and 2, the *LSAT* problem and the path plan feasibility problem constructed are equivalent.

We note that the NP-completeness result of our path plan feasibility problem is similar to that of the flow control problem in store-and-forward packet switching networks [26]. However, there is an important difference. In the store-and-forward network, packets are removed from the network after reaching their destinations. This allows different packets to share the same destination. Indeed, the problem instances used for reductions in the NP-completeness proofs have packets sharing the same destination [26]. In contrast, we assume that the goal locations of all agents are distinct because agents will stay at their goal locations after reaching them. Hence, the proofs of [26] are not directly applicable to our problem.

4.3. Feasibility test

We now develop feasibility test algorithms for path plans. We start with a basic brute-force algorithm (Section 4.3.1) and then incorporate heuristics (Section 4.3.2) and optimization techniques (Sections 4.3.3 and 4.3.4) to enhance algorithm efficiency.

4.3.1. A basic algorithm

Based on Theorem 1, we design a feasibility test (Algorithm 1). The main idea is to look for an acyclic complete LDG by a depth-first search of combinations of unsettled edges. Suppose that the partial LDG is acyclic. The algorithm repeatedly scans through the unsettled edge pairs ($\mathcal{E}_0$, line 1) and incrementally selects edges from them to produce complete LDGs. The edges selected, called *settled edges*, are recorded in the set $\mathcal{E}_{\text{settled}}$. The test involves a recursive function `FeasibilityTest` that checks whether there exists an acyclic complete LDG based on an input $\mathcal{E}_{\text{settled}}$ (line 5). The feasibility of a path plan is derived by a function call with an empty $\mathcal{E}_{\text{settled}}$ (line 3). The initial LDG in a function call is generated by adding the input $\mathcal{E}_{\text{settled}}$ to the partial LDG (line 6). Then, the function attempts to further select edges from the remaining unsettled edge pairs.

When examining an unsettled edge pair, there are three possible cases: (1) adding either unsettled edge will form a cycle in the LDG; (2) adding one unsettled edge will form a cycle in the LDG, but adding the other edge will not; (3) adding either unsettled edge will not form a cycle in the LDG. In case (1), the function concludes that no acyclic complete LDG exists based on the input $\mathcal{E}_{\text{settled}}$ (lines 19-20). In case (2), the function adds the edge not forming a cycle to $\mathcal{E}_{\text{settled}}$ and the LDG (lines 21-23). Both cases (1) and (2) cut the search space substantially and hence should be prioritized. Thus, in the first function call (by line 3), we skip lines 11-17 (due to line 10) and scan all unsettled edge pairs to find cases (1) and (2) (lines 18-23). After that, a recursive function call is made (line 24), in which we select an arbitrary unsettled edge pair $\langle e, e' \rangle$ and check if it falls in case (3) (lines 11-12).[2] If so, we first add e to $\mathcal{E}_{\text{settled}}$ and make a recursive call to see if an acyclic complete LDG can be produced (lines 13-14). If this is so, the function returns a positive result (line 15). Otherwise, we perform backtracking by removing e, adding e' to $\mathcal{E}_{\text{settled}}$ and making another recursive call (lines 16-17). Only when both calls return negative results, the function concludes that no acyclic complete LDG exists. If the

[2] In our implementation, unsettled edge pairs are ordered by the 4-tuple of agent identifiers and node indexes involved on their paths. We choose the first edge pair in line 11.

unsettled edge pair $\langle e, e' \rangle$ selected falls in case (1) or (2), we scan all unsettled edge pairs again to find cases (1) and (2) (lines 18-23), and then make another recursive call (line 24).

Algorithm 1: Feasibility test of path plan $\mathcal{P}$.

```
1  LDG_0 ← the initial partial LDG of P;
2  E_0 ← unsettled edge pairs in P;
3  return FeasibilityTest(∅);
4
5  Function FeasibilityTest(E_settled):
6      LDG ← add settled edges in E_settled to LDG_0;
7      if |E_settled| = |E_0| then
8          return feasible (with complete LDG);
9      E_unsettled ← remove unsettled edge pairs that contain settled edges in E_settled from E_0;
10     if this is not the first FeasibilityTest call then
11         Select a pair of edges ⟨e, e'⟩ from E_unsettled;
12         if neither of e and e' forms a cycle in LDG then
13             Add e to E_settled;
14             if FeasibilityTest(E_settled) is feasible then
15                 return feasible (with complete LDG);
16             Remove e from E_settled and add e' to E_settled;
17             return FeasibilityTest(E_settled);
18     foreach pair of unsettled edges ⟨e, e'⟩ in E_unsettled do
19         if e and e' both form a cycle in LDG then
20             return infeasible (together with the two agents that e and e' involve);
21         else if one of e and e' forms a cycle in LDG then
22             ē ← the edge not forming a cycle in LDG;
23             Add ē to LDG and ē to E_settled;
24     return FeasibilityTest(E_settled);
```

Theorem 3. *Algorithm 1 is sound and complete.*

Proof. Each recursive function call by lines 14 and 17 increases the number of settled edges $|\mathcal{E}_{\text{settled}}|$ by at least one. This is also true for each recursive function call by line 24 (except that in the first function call), because in order for lines 18-23 to be executed, the unsettled edge pair selected by line 11 must fall in case (1) or (2). The recursion terminates when $|\mathcal{E}_{\text{settled}}|$ reaches $|\mathcal{E}_0|$ (lines 7-8). Thus, the function always returns a result, so Algorithm 1 is complete.

We prove that Algorithm 1 is sound by backward induction on the function call. Note that the initial LDG generated by the input $\mathcal{E}_{\text{settled}}$ (line 6) is guaranteed to be acyclic in any function call.

1) Base case: when $|\mathcal{E}_{\text{settled}}| = |\mathcal{E}_0|$ (line 7), it means that there is no unsettled edge pair left. A positive result (feasible) is returned (line 8) because the LDG is complete and acyclic.

2) Induction: we show that the function result is correct for an input $\mathcal{E}_{\text{settled}}$, given the hypothesis that the result is correct for any input $\mathcal{E}'_{\text{settled}}$ such that $|\mathcal{E}_{\text{settled}}| < |\mathcal{E}'_{\text{settled}}|$. We examine all return clauses in the function except line 8, which is covered in the base case. In line 20, selecting either edge from an unsettled edge pair forms a cycle in the LDG, so a negative result is returned. In line 24, all newly added settled edges by lines 21-23 have to be selected to avoid cycles, so the problems before and after adding these settled edges are equivalent. Since the recursive call is correct by the induction hypothesis, the result returned in line 24 is correct. In lines 13-17, the function joins the results of two recursive calls (correct by the induction hypothesis) with different selections of settled edges from the same unsettled edge pair. If both results are negative, a negative result is returned because selecting either edge cannot lead to an acyclic complete LDG. Otherwise, at least one acyclic complete LDG is returned so that the path plan is feasible. □

4.3.2. Heuristic feasibility test

The feasibility test (Algorithm 1) has exponential time complexity in the worst case, as it may need to exhaustively check all possible complete LDGs. However, in practice, the test typically runs efficiently when the number of agents is moderate, as shall be demonstrated in our experiments. This efficiency arises because the algorithm first resolves the edge pairs in cases (1) and (2) before addressing those in case (3). In addition, the test terminates as soon as an acyclic complete LDG is found.

However, as the number of agents increases, it is not surprising that the computational time of Algorithm 1 grows fast. To address this issue, we design a faster heuristic feasibility test as shown in Algorithm 2.

Algorithm 2: Heuristic feasibility test of path plan $\mathcal{P}$.

```
1  LDG ← the initial partial LDG of P;
2  E_unsettled ← unsettled edge pairs in P;
3  while E_unsettled ≠ ∅ do
4      foreach pair of unsettled edges ⟨e, e'⟩ in E_unsettled do
5          if e and e' both form a cycle in LDG then
6              return infeasible (together with the two agents that e and e' involve);
7          else if one of e and e' forms a cycle in LDG then
8              ē ← the edge not forming a cycle in LDG;
9              Add ē to LDG;
10             break;
11         else /* neither of e and e' forms a cycle in LDG                    */
12             Add e to LDG;
13         Remove ⟨e, e'⟩ from E_unsettled;
14     foreach pair of unsettled edges ⟨e, e'⟩ in E_unsettled do
15         if e and e' both form a cycle in LDG then
16             return infeasible (together with the two agents that e and e' involve);
17         else if one of e and e' forms a cycle in LDG then
18             ē ← the edge not forming a cycle in LDG;
19             Add ē to LDG;
20         Remove ⟨e, e'⟩ from E_unsettled;
21 return feasible (with complete LDG);
```

In essence, lines 11-17 in Algorithm 1 is replaced by a for-loop (lines 4-13) in Algorithm 2, which is quite similar to lines 18-23 in Algorithm 1. We still need to handle the three possible cases when examining an unsettled edge pair in the first for-loop (lines 4-13) of Algorithm 2. In case (1), if selecting either edge forms a cycle in the LDG, a negative result is returned (lines 5-6). In case (2), after adding the edge that does not form a cycle to $\mathcal{E}_{\text{settled}}$ and the LDG, the loop breaks (lines 7-10). Hence, at most one edge from case (2) is added during the loop. In case (3), if selecting either edge does not form a cycle, we only attempt to select one of them (lines 11-12), and the other edge will not be selected in the heuristic feasibility test.

The key difference between the heuristic feasibility test and the original one is that the heuristic approach continuously selects edges in case (3) whenever no case (2) is found. When case (2) is found, the first for-loop breaks (lines 7-10) and the execution goes into the second for-loop (lines 14-20) of Algorithm 2 (same as lines 18-23 of Algorithm 1) which scans all unsettled edge pairs to exhaust cases (1) and (2). The rationale is that cases (1) and (2) are more restrictive in edge selection than case (3), so prioritizing them can enhance efficiency. After exhausting cases (1) and (2), the execution goes back to the first for-loop to deal with case (3) again. Since only one edge is selected for testing in case (3), Algorithm 2 is no longer a recursive algorithm and there is no backtracking. This results in substantial improvement in computational speed. However, the heuristic feasibility test can produce false negative results. It is possible for the original feasibility test to return a feasible outcome, while the heuristic test deems it infeasible. This occurs because, in case (3), if the heuristic test selects one edge from an unsettled edge pair but constructing an acyclic complete LDG requires selecting the other edge, the test will fail to find the latter and hence return an infeasible outcome.

To avoid false negative results in our solution framework, we implement a two-level feasibility test. If the heuristic feasibility test returns an infeasible outcome, we then run the original feasibility test to validate the outcome. Our experiments show that the two-level feasibility test runs faster than the original feasibility test, particularly as the number of agents increases.

4.3.3. Check for cycles in LDG

A fundamental operation of the feasibility test is to examine if adding an unsettled edge will form a cycle in the LDG (e.g., lines 12, 19 and 21 of Algorithm 1). To check whether adding an unsettled edge $\langle v_{i,j}, v_{i',j'} \rangle$ will form a cycle in the LDG, we can simply run a breadth-first search to inspect if there is a path from $v_{i',j'}$ to $v_{i,j}$ before the edge is added. The time complexity of the search is $\mathcal{O}(|\mathcal{V}_{\text{LDG}}| + |\mathcal{E}_{\text{LDG}}|)$. Given that thousands of such checks are performed in a single feasibility test, especially as the number of agents increases and the LDG becomes larger, the computational efficiency of the test can be significantly impacted by the efficiency of these checks.

Exploiting the characteristic of the LDG that all location states of an agent form a linear structure based on the path of the agent (i.e., the first part of $\mathcal{E}_{\text{LDG}}$), we develop a more efficient algorithm to determine whether a path exists in the LDG. To facilitate

presentation, a location state $v_{i',j'}$ is said *reachable* from another location state $v_{i,j}$ if a directed path exists from $v_{i,j}$ to $v_{i',j'}$ in the LDG. We use a 2-dimensional connection array $C[i,j][i'] = j^*$ to record the lowest-indexed location state v_{i',j^*} of agent $a_{i'}$, reachable from the location state $v_{i,j}$ of agent a_i. To check if a path already exists from $v_{i,j}$ to $v_{i',j'}$, we simply compare $C[i,j][i']$ with j': if $C[i,j][i'] \leq j'$, a path from $v_{i,j}$ to $v_{i',j'}$ exists so that adding an edge $\langle v_{i',j'}, v_{i,j}\rangle$ will form a cycle in the LDG; otherwise, if $C[i,j][i'] > j'$, a path from $v_{i,j}$ to $v_{i',j'}$ does not exist and thus adding an edge $\langle v_{i',j'}, v_{i,j}\rangle$ will not form a cycle in the LDG. Note that by definition, the connection array has the characteristic that $C[i,j][k] \leq C[i,x][k]$ for any i, $1 \leq j \leq x \leq n_i$ and k. This is because the location state $v_{i,x}$ can be reached from $v_{i,j}$ following the path of agent a_i. Thus, the lowest-indexed location state reachable from $v_{i,x}$ can also be reached from $v_{i,j}$.

For example, in the partial LDG of Fig. 5 (upper) with predetermined edges marked by solid lines only, the lowest-indexed location states of agent a_3 reachable from the location states $v_{2,1}$, $v_{2,2}$, $v_{2,3}$, $v_{2,4}$, $v_{2,5}$ and $v_{2,6}$ of agent a_2 are $v_{3,2}$, $v_{3,2}$, nil, nil, nil and nil respectively. Hence, $C[2,1][3] = C[2,2][3] = 2$ and $C[2,3][3] = C[2,4][3] = C[2,5][3] = C[2,6][3] = +\infty$. Since $C[2,4][3] = +\infty > 5$, no path exists from $v_{2,4}$ to $v_{3,5}$. Therefore, adding the unsettled edge $\langle v_{3,5}, v_{2,4}\rangle$ will not form a cycle in the LDG.

All entries in the connection array C are initially set to $+\infty$, with the exception that for each agent $a_i \in \mathcal{A}$ and $1 \leqslant j \leqslant n_i$, we set $C[i,j][i] = j$, as the lowest-indexed location state of a_i reachable from $v_{i,j}$ is itself. As edges are added to the LDG, the entries in the connection array C are updated and their values can only decrease when updated. Note that the entries $C[i,j][i]$ never need to be updated because if they decrease, we have $C[i,j][i] < j$, i.e., from $v_{i,j}$, we can reach a lower-indexed location state of a_i, which means that a cycle is formed. When an edge $\langle v_{i,j}, v_{i',j'}\rangle$ is added to the LDG without forming any cycle, the connection array is updated as shown in Algorithm 3. If the lowest-indexed location state of $a_{i'}$ reachable from $v_{i,j}$ is already prior to $v_{i',j'}$, no update is needed since no new connection is formed in the LDG (lines 1-2). Otherwise, two updates are made to the connection array.

First, we update the connection array entries for each agent $a_k \in \mathcal{A} \setminus \{a_i\}$ reachable from agent a_i's location states $v_{i,x}$ that are prior to or equal to $v_{i,j}$ (i.e., $1 \leqslant x \leqslant j$). If $C[i,x][k] > C[i',j'][k]$, since a path $v_{i,x} \to \cdots \to v_{i,j} \to v_{i',j'} \to \cdots \to v_{k,C[i',j'][k]}$ is now formed, we update the lowest-indexed location state of a_k reachable from $v_{i,x}$ to $C[i',j'][k]$ (lines 5-6). In addition, if there exists an x_0 where $1 < x_0 \leqslant j$ such that $C[i,x_0][k] \leqslant C[i',j'][k]$, we can infer that $C[i,x][k] \leqslant C[i,x_0][k] \leqslant C[i',j'][k]$ for each $x < x_0$. In this case, we can skip updating $C[i,x][k]$ for all $x < x_0$ (lines 7-8). For example, in the partial LDG of Fig. 5 (upper), the lowest-indexed location states of agent a_2 reachable from the location states $v_{3,1}$, $v_{3,2}$, $v_{3,3}$, $v_{3,4}$, $v_{3,5}$ and $v_{3,6}$ of agent a_3 are $v_{2,6}$, $v_{2,6}$, $v_{2,6}$, $v_{2,6}$, $v_{2,6}$ and nil respectively. Hence, $C[3,1][2] = C[3,2][2] = C[3,3][2] = C[3,4][2] = C[3,5][2] = 6$ and $C[3,6][2] = +\infty$. When the edge $\langle v_{3,5}, v_{2,4}\rangle$ is added, we examine the entries $C[3,j][2]$ where $1 \leq j \leq 5$. Since $C[3,j][2] = 6 > 4 = C[2,4][2]$, $C[3,j][2]$ is updated the value of $C[2,4][2] = 4$, meaning that the lowest-indexed location state of agent a_2 reachable from the location states $v_{3,1}$, $v_{3,2}$, $v_{3,3}$, $v_{3,4}$ and $v_{3,5}$ becomes $v_{2,4}$.

Second, we update the connection array entries reachable from agents other than a_i and $a_{i'}$. For all the location states $v_{k,x}$ ($1 \leqslant x \leqslant n_k$) of each agent $a_k \in \mathcal{A} \setminus \{a_i, a_{i'}\}$, if $C[k,x][i] \leq j$ (meaning that there was a path from $v_{k,x}$ to $v_{i,j}$), we update the lowest-indexed location state for all agents $a_m \in \mathcal{A} \setminus \{a_k\}$ reachable from $v_{k,x}$. If $C[k,x][m] > C[i',j'][m]$, we update the lowest-index location state of a_m reachable from $v_{k,x}$ to $C[i',j'][m]$ (lines 11-14), because a path $v_{k,x} \to \cdots \to v_{i,C[k,x][i]} \to \cdots \to v_{i,j} \to v_{i',j'} \to \cdots \to v_{m,C[i',j'][m]}$ is now formed. Similarly, if $C[k,x_0][i] > j$ for some x_0 (no path existed from v_{k,x_0} to $v_{i,j}$), we must have $C[k,x][i] \geq C[k,x_0][i] > j$ for any $x > x_0$ (no path from $v_{k,x}$ to $v_{i,j}$ could exist either). In this case, we can skip the updates for all $x > x_0$ (lines 15-16). For example, in the partial LDG of Fig. 5 (upper), the lowest-indexed location states of agent a_1 reachable from the location states $v_{3,1}$, $v_{3,2}$, $v_{3,3}$, $v_{3,4}$, $v_{3,5}$ and $v_{3,6}$ of agent a_3 are all nil. Hence, $C[3,1][1] = C[3,2][1] = C[3,3][1] = C[3,4][1] = C[3,5][1] = C[3,6][1] = +\infty$. These entries do not change after the edge $\langle v_{3,5}, v_{2,4}\rangle$ is added. When the edge $\langle v_{2,4}, v_{1,3}\rangle$ is further added, since the entries $C[3,j][2] = 4$ for $1 \leq j \leq 5$, we examine the corresponding entries $C[3,j][1]$. Since $C[3,j][1] = +\infty > 3 = C[1,3][1]$, $C[3,j][1]$ is updated the value of $C[1,3][1] = 3$, meaning that the lowest-indexed location state of agent a_1 reachable from the location state $v_{3,1}$, $v_{3,2}$, $v_{3,3}$, $v_{3,4}$ and $v_{3,5}$ becomes $v_{1,3}$.

The worst-case time complexity of updating the connection array (Algorithm 3) is $\mathcal{O}(|\mathcal{A}||\mathcal{V}_{\text{LDG}}|)$, while the time complexity of querying a path in the LDG is $\mathcal{O}(1)$. Although the update complexity can be higher than that of a breadth-first search, the number of updates is often significantly lower than the number of path queries in a single call to `FeasibilityTest` in Algorithm 1, because unsettled edge pairs in case (3) (adding either unsettled edge will not form a cycle in the LDG) do not lead to any update to the LDG (lines 18-23 of Algorithm 1). Therefore, checking cycles with the connection array can significantly reduce the overall computational time of Algorithm 1. Note that Algorithm 3 is one-way, meaning that the update cannot be canceled if an edge is removed. To address this, our implementation backs up the connection array before performing an update if there is a possibility of reverting it later (line 16 of Algorithm 1). Since the required space for the connection array is relatively small ($\mathcal{O}(|\mathcal{A}||\mathcal{V}_{\text{LDG}}|)$), the space and time spent on backup and recovery operations are both negligible.

4.3.4. Group agents in LDG

We present a grouping technique to further enhance the computational efficiency of feasibility tests motivated by the following observation. If there is no predetermined edge between different agents (i.e., in the second and third parts of $\mathcal{E}_{\text{LDG}}$), a very simple method can be used to select the unsettled edges and construct an acyclic complete LDG. First, we assign distinct priorities to all agents (e.g., in the order of their indexes). For each unsettled edge pair, we select the edge that starts from the higher-priority agent and ends at the lower-priority agent. Since no edge is directed from a lower-priority agent to a higher-priority one, the resulting complete LDG must be free of cycles.

If there are predetermined edges between agents, we can attempt to assign priorities to the agents following predetermined edges. Specifically, we construct an agent dependency graph (Agent DG). In the Agent DG, the nodes represent agents, and the edges are

Algorithm 3: Update C when $\langle v_{i,j}, v_{i',j'} \rangle$ is added.

```
if C[i, j][i'] ≤ j' then
    return;
for a_k ∈ A \ {a_i} do
    for x ← j downto 1 do
        if C[i, x][k] > C[i', j'][k] then
            C[i, x][k] ← C[i', j'][k];
        else
            break;
for a_k ∈ A \ {a_i, a_i'} do
    for x ← 1 to n_k do
        if C[k, x][i] ≤ j then
            for a_m ∈ A \ {a_k} do
                if C[k, x][m] > C[i', j'][m] then
                    C[k, x][m] ← C[i', j'][m];
        else
            break;
```

created based on the predetermined edges in the LDG. For each predetermined edge from agent a_i to agent a_j in the LDG, we add a corresponding edge $\langle a_i, a_j \rangle$ in the Agent DG. If the Agent DG is acyclic, we can derive a topological ordering of all agents in the Agent DG. Then, we can assign decreasing priorities to agents according to the topological ordering, which would guarantee that every predetermined edge points from a higher-priority agent to a lower-priority agent. As a result, the above selection method of unsettled edges can still be applied to construct an acyclic complete LDG.

If there are cycles in the Agent DG, we can assign the same priority to every agent involved in a cycle and group all agents of the same priority together. Specifically, a strongly connected component in the Agent DG naturally corresponds to a group of agents to be assigned the same priority. We use Tarjan's algorithm [27] to divide the Agent DG into strongly connected components (agent groups). Tarjan's algorithm performs a depth-first search (DFS) on a directed graph. Nodes are pushed onto a stack as they are first visited. Each node v is assigned a unique integer $v.\text{number}$ which numbers the nodes consecutively in the order of their first visits in the DFS. Each node v also maintains a value $v.\text{lowlink}$ that represents the smallest index of any node on the stack known to be reachable from v in the DFS. If $v.\text{number} = v.\text{lowlink}$ after exploring all v's neighbors in the DFS, all nodes above v on the stack together with v form a strongly connected component and they are popped from the stack. After running Tarjan's algorithm, we assign a distinct priority to each agent group following a topologial ordering of all agent groups, so that every predetermined edge in the LDG points from a higher-priority group to a lower-priority group. For each unsettled edge pair between agents in different groups, we also select the edge pointing from the higher-priority group to the lower-priority group. In this way, there will be no cycle involving more than one agent group. In the feasibility test, we only need to consider unsettled edge pairs between agents within the same group. Furthermore, the unsettled edge pairs in each group can be handled separately, as the selection of edges within one group does not affect the edge selection in another group, since the selection of cross-group edges based on group priority ensures that no cycle can be formed in the LDG between different groups. Thus, the path plan feasibility problem is reduced to those in respective agent groups. Hence, grouping agents in the Agent DG can reduce the number of unsettled edge pairs to be handled by the feasibility test (Algorithm 1).

Fig. 5 illustrates an example LDG for four agents and its corresponding Agent DG. In the LDG, predetermined edges are represented by solid lines and consist of the first three parts of $\mathcal{E}_{\text{LDG}}$. The predetermined edges colored red (e.g., $\langle v_{1,2}, v_{2,2} \rangle$) correspond to the second part of $\mathcal{E}_{\text{LDG}}$, related to the initial locations of agents, while the predetermined edges colored blue (e.g., $\langle v_{3,5}, v_{2,6} \rangle$) represent the third part of $\mathcal{E}_{\text{LDG}}$, related to the goal locations of agents. As seen in the corresponding Agent DG, $\{a_1\}$ and $\{a_2, a_3, a_4\}$ form two strongly connected components, grouping the agents accordingly. We can assign a higher priority to group $\{a_1\}$ and a lower priority to group $\{a_2, a_3, a_4\}$, since there is a path from the former to the latter in the Agent DG. This allows us to select unsettled edges between a_1 and other agents directly, since they belong to different groups. For the unsettled edge pair $\langle v_{1,4}, v_{2,3} \rangle$ and $\langle v_{2,4}, v_{1,3} \rangle$, we select the edge $\langle v_{1,4}, v_{2,3} \rangle$ because a_1 has a higher priority than a_2. The remaining unsettled edge pairs cannot be resolved at this point and should be passed to Algorithm 1 to complete the feasibility test.

The agent grouping technique preserves the completeness of feasibility tests. On the one hand, if Algorithm 1 finds an acyclic LDG within each agent group, then together with the selection of cross-group unsettled edges, we can obtain an acyclic complete LDG for all agents, which demonstrates that the original path plan of all agents is feasible. On the other hand, if Algorithm 1 cannot find any acyclic LDG in some agent group, it means that the path plan of the agents within that group is infeasible, which in turn implies that the original path plan of all agents is infeasible.

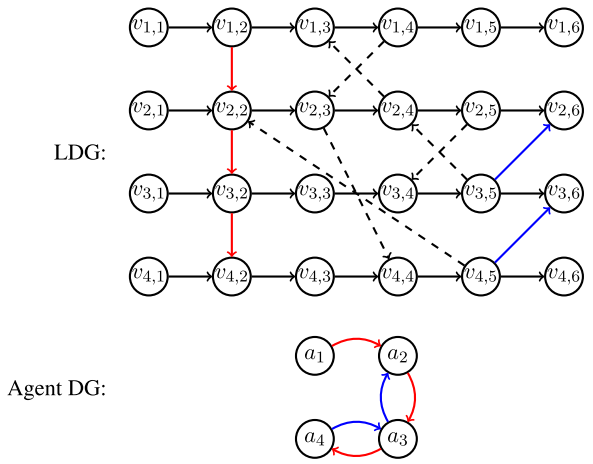


Fig. 5. Grouping the agents in a LDG (upper) with its Agent DG (lower).

5. Coordinating path execution

Assuming that the initial path plan is feasible (which naturally holds for the output of any MAPF solver), we now present our solution to coordinate the *move* and *wait* actions taken by agents during the execution and ensure that all agents can eventually arrive at their goal locations. Our solution uses the feasibility test of path plans (Section 4.3) as a component. Instead of simply executing an acyclic complete LDG precomputed from the initial path plan, we invoke feasibility tests at runtime to dynamically decide which agents can take *move* actions based on their location states while avoiding conflicts and deadlocks. In this paper, we focus on a centralized solution where the agents can sense changes in their location states and send updates to a central controller. Developing distributed solutions is left for future work.

In the execution, we partition all agents into two sets: unblocked agents U (those taking *move* actions) and blocked agents B (those taking *wait* actions). Remember that a *move* action cannot be canceled before the agent arrives at the adjacent node. Thus, an unblocked agent remains in U until it arrives at the next node on its path. Upon its arrival, we need to decide whether to block the agent from taking a further *move* action. If multiple agents in U arrive at the next nodes on their respective paths at the same timestep, decisions need to be made for all these agents. In addition, we need to decide whether to unblock any agents in B and allow them to take *move* actions. For simplicity, we assume that an agent arriving at the next node on its path is moved from U to B by default. Then, given the current partitions U and B, we focus on designing an algorithm to determine which agents in B to unblock.

If we unblock a set of agents $\Delta U \subseteq B$, the partitions become $U \cup \Delta U$ and $B \setminus \Delta U$. Recall that a blocked agent occupies its current node only, while an unblocked agent occupies both nodes connected by an edge. To guarantee conflict-freeness, we require that the nodes occupied by different agents are all distinct. To ensure deadlock-freeness, we require that if all agents in $U \cup \Delta U$ arrive at their next nodes, the *residual* path plan is feasible. Note that owing to uncertainty, we do not know when these agents will arrive at their next nodes. If the above requirement is satisfied, it also guarantees that the residual path plan is feasible if any subset of the agents in $U \cup \Delta U$ arrive at their next nodes, because we can always wait until the remaining agents in $U \cup \Delta U$ arrive at their next nodes before unblocking any new agent.

For example, in Fig. 6 (left), if all four agents are unblocked initially and arrive at the next nodes on their paths, none of them can move any further since a deadlock is formed. As shown in Fig. 6 (right), there is a cycle in the (partial) LDG of the residual path plan, so it is infeasible. Thus, at most three agents can be unblocked initially.

To maximize the path execution effectiveness, the objective of our algorithm is to maximize $|\Delta U|$ (the number of agents to unblock) while meeting the above requirements. In what follows, we present different algorithms to determine which agents to unblock. These algorithms are invoked at the timesteps when at least one agent in U completes its *move* action (arriving at the next node on its path) and is moved from U to B. We remark that an acyclic complete LDG returned from the feasibility test is not really required for path execution in our framework (only the binary output feasible/infeasible is used in choosing agents to unblock). The acyclic complete LDG returned is only used as hints (on which unsettled edge in a pair to test first) to accelerate the feasibility tests in the next invocation of the algorithm to unblock agents. The agents taking *move* actions are always decided by the following algorithms to unblock agents.

5.1. Naive approach

A naive approach (Algorithm 4) is to iterate through all subsets of B in decreasing order of size and check each subset against the above requirements. Once a subset satisfying the requirements is found, it gives the largest set of agents to unblock. However, the number of all possible subsets of B is $2^{|B|}$, which means that in the worst case, this approach needs to conduct an exponential

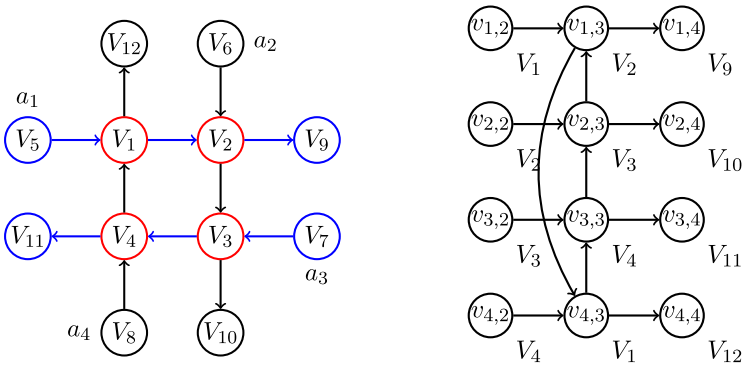


Fig. 6. A path plan (left) and the LDG of the residual path plan if all four agents are unblocked initially (right).

number of feasibility tests. Even if the largest set of agents to unblock has a size i close to $|B|$, we still need to perform at least $\binom{|B|}{i+1} = \frac{|B|!}{(i+1)!(|B|-i-1)!} \approx |B|^{|B|-i-1}$ feasibility tests.

Algorithm 4: Naive approach to unblock agents.

```
1 for i = |B| downto 1 do
2     foreach ΔU ⊆ B where |ΔU| = i do
3         P ← the residual paths of the agents A if all agents in U ∪ ΔU arrive at their next nodes;
4         if (P is feasible) then
5             return ΔU;
```

5.2. Semi-naive approach

Instead of considering all agents in B, some agents can definitely be unblocked or must be kept blocked. We can first identify these agents (Algorithm 5) and then decide the remaining agents in B with the naive approach.

First, for each agent $a_i \in B$, if the next node v on its path is not occupied and does not appear in the residual path of any other agent, a_i can definitely be unblocked (lines 3-6).

Second, for each agent $a_i \in B$, if the next node v on its path is currently occupied by another agent a_j, a_i cannot be unblocked. This occurs when a_j is taking a *wait* action at node v, or a_j is taking a *move* action from v to an adjacent node, or a_j is taking a *move* action from an adjacent node to v. All identified agents a_i to keep blocked are recorded in the set B_K (lines 7-8).

Next, for each agent $a_i \in B$, if the next node v on its path is its goal location and v also appears in the residual path of another agent a_j, a_i cannot be unblocked. This is because if a_i arrives at v first, it will occupy v infinitely and prevent a_j from passing through v. All identified agents a_i to keep blocked are recorded in the set B_K (lines 9-10).

Finally, we set $B' = B \setminus B_K$ and examine whether these remaining agents can be unblocked with the naive approach by substituting B with B' in Algorithm 4.

The semi-naive approach reduces the size of the blocked agent set B, while still providing the optimal solution to maximize the size of the unblocked agent set. However, it encounters the same issue as the naive approach when the number of agents increases because B' can remain quite large in such cases.

5.3. Heuristic approach

To further optimize the computational efficiency of coordination, we develop a heuristic algorithm (Algorithm 6) to determine which agents in B can be unblocked. This algorithm runs much faster than both the naive and semi-naive approaches. Although the heuristic approach does not guarantee an optimal solution, empirical results show that it performs nearly as well as the optimal approaches (naive and semi-naive).

Before the execution of Algorithm 6, we run Algorithm 5 to identify agents that can definitely be unblocked or must be kept blocked, and set $\Delta U = B \setminus B_K$. Then we set the initial location states of all agents in $U \cup \Delta U$ to the next nodes on their respective paths and run the feasibility test. In our implementation, to improve efficiency, the feasibility test uses the previously found acyclic complete LDG (in the last run of Algorithm 6) as hints for choosing which edge to test first from a pair of unsettled edges (line 13 of Algorithm 1 and line 13 of Algorithm 2). If the residual path plan is feasible, we can safely unblock the agents ΔU (lines 3-5). Otherwise, the feasibility test must return two agents involved in the cycles found in the LDG (line 20 of Algorithm 1). Typically, between these two agents, at least one agent a comes from ΔU, since the residual path plan before unblocking ΔU was feasible. In

Algorithm 5: Identify agents that can definitely be unblocked or must be kept blocked.

```
1  B_K ← ∅;
2  foreach agent a_i ∈ B do
3      v ← the next node on a_i's path;
4      if v is not occupied and does not appear in the residual path of any other agent then
5          U ← U ∪ {a_i};
6          B ← B \ {a_i};
7      if v is occupied by another agent then
8          B_K ← B_K ∪ {a_i};
9      if v is a_i's goal location and v appears in the residual path of another agent then
10         B_K ← B_K ∪ {a_i};
11 B' ← B \ B_K;
```

this case, we remove a from ΔU to break cycles and check the feasibility of the residual path plan again. This process is repeated until the residual path plan becomes feasible (lines 2-7). If both agents returned from the feasibility test are not in ΔU, we simply set ΔU to an empty set (lines 8-9).

In the special case that both U and the resultant ΔU are empty, we need to pick at least one agent to unblock in order to proceed with the path execution. To do so, we examine each agent in $B \setminus B_K$ and check whether unblocking it alone leaves the residual path plan feasible (lines 11-15). Since the path plan is feasible, at least one agent can be unblocked.

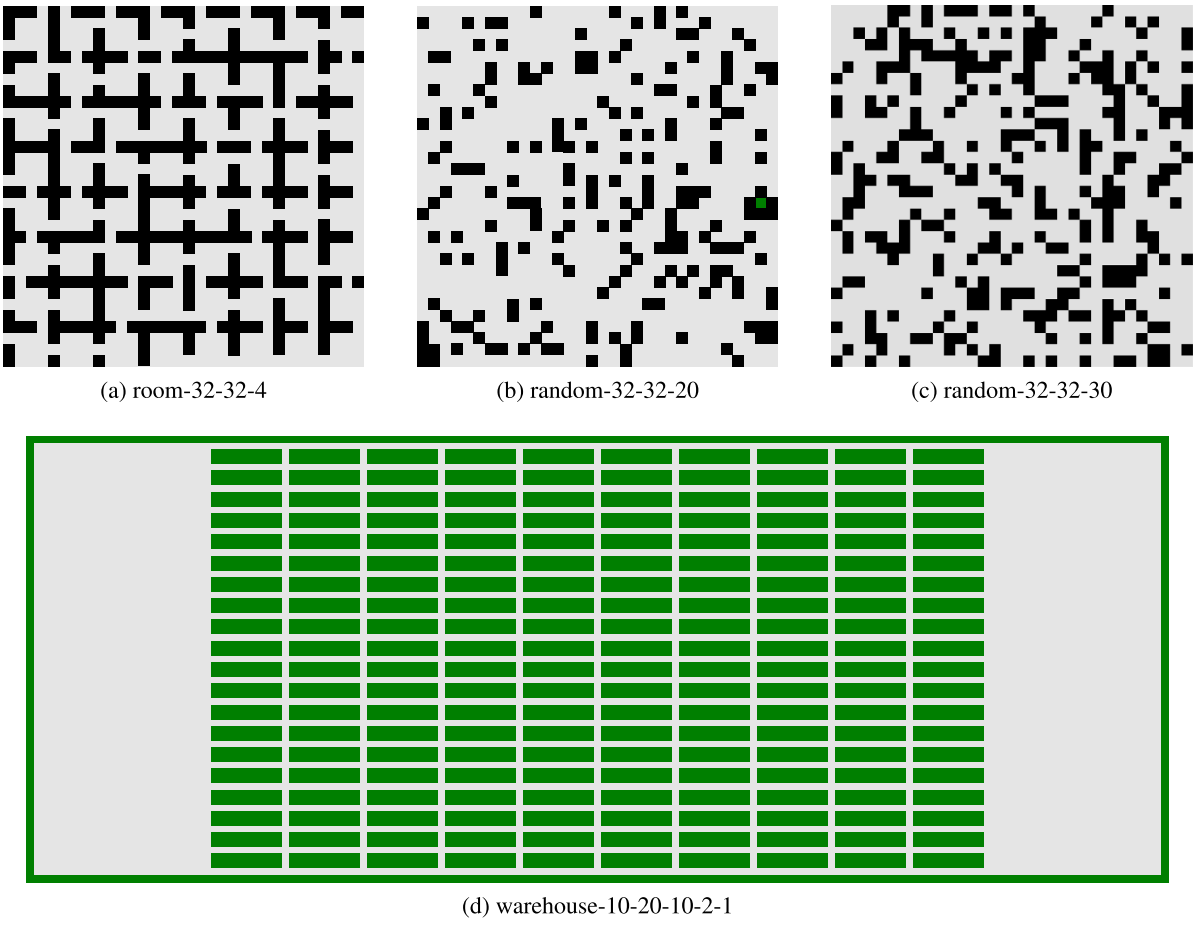
Algorithm 6: Heuristic approach to unblock agents.

```
1  ΔU ← B \ B_K;
2  while ΔU ≠ ∅ do
3      P ← the residual paths of the agents A if all agents in U ∪ ΔU arrive at their next nodes;
4      if (P is feasible) then
5          break;
6      if an agent returned from feasibility test is in ΔU then
7          remove the agent from ΔU;
8      else
9          ΔU ← ∅;
10 if U ∪ ΔU = ∅ then
11     foreach agent a_i in B \ B_K do
12         P ← the residual paths of the agents A if all agents in U ∪ {a_i} arrive at their next nodes;
13         if (P is feasible) then
14             ΔU ← {a_i};
15             break;
16 return ΔU;
```

Algorithm 6 ensures that (1) agents taking *move* actions do not collide with each other or with agents taking *wait* actions; and (2) if all or any subset of *move* actions being executed are completed, the residual path plan remains feasible. Thus, running Algorithm 6 guarantees that the path execution is conflict-free and deadlock-free.

6. Experimental evaluation

We conduct experiments on four different maps. The first map is room-32-32-4 (Fig. 7a), which consists of 8 rows and 8 columns of 3×3 rooms separated by walls, with 1-unit-wide corridors (doors) connecting the rooms. This map has more shared locations along agent paths compared to other maps of similar size, as agents must pass through these narrow corridors. The second map is warehouse-10-20-10-2-1 (Fig. 7d), simulating the layout of a real warehouse with storage racks. The third map is random-32-32-20 (Fig. 7b), where 20% cells of the grid map are randomly positioned obstacles, but it is guaranteed to be connected so that any location can be reached from any other location. The above three maps are taken from the Moving AI MAPF benchmark [4]. The fourth map random-32-32-30 (Fig. 7c, generated by us) is similar to random-32-32-20 but with 30% cells of the grid map being randomly positioned obstacles.



(a) room-32-32-4 (b) random-32-32-20 (c) random-32-32-30

(d) warehouse-10-20-10-2-1

Fig. 7. Maps from the moving AI MAPF benchmark [4] and generated by us.

We use the even scenario files from the Moving AI MAPF benchmark, which have an even mix of short and long agent paths, to assign the start and goal locations of agents in the first three maps, and we create similar scenario files for the fourth map. Given M agents, we use the first M lines from each scenario file to construct a problem instance. For each instance generated, we run Explicit Estimation CBS (EECBS) [28] with the suboptimality factor set to 1.1 to obtain a near-optimal MAPF solution under the standard assumption that each *move* action takes one timestep to complete. EECBS is a state-of-the-art variant of Conflict-Based Search (CBS) [29] that incorporates many techniques for speeding up CBS. We note that the MAPF solution produced by EECBS may have following conflicts, while our execution does not allow following conflicts (as discussed in Section 3). To address it, we make use of our feasibility test (Section 4.3) to convert the EECBS solution into a plan without following conflicts before execution. Specifically, we first extract the path of each agent (the trajectory from the start location to the goal location) from the EECBS output and construct the LDG with predetermined and unsettled edges (Section 4.1). Then, we feed the LDG into the feasibility test (Algorithm 1) to compute an acyclic complete LDG (i.e., determine the order for the agents to occupy common nodes). Recall that in the feasibility test, if neither unsettled edge in a pair forms a cycle in the LDG (line 12), we proceed by randomly selecting an unsettled edge to test first (line 13). To follow the EECBS solution as closely as possible here, we use as hints the timestamps in the EECBS output for choosing which unsettled edge to test first. Specifically, for a pair of unsettled edges $\langle v_{i,j+1}, v_{i',j'} \rangle$ and $\langle v_{i',j'+1}, v_{i,j} \rangle$ (where the location states $v_{i,j}$ and $v_{i',j'}$ refer to a common node in the map), if agent a_i achieves $v_{i,j}$ before agent a_i achieves $v_{i',j'}$ in the EECBS output, we select the edge $\langle v_{i,j+1}, v_{i',j'} \rangle$ first; otherwise, we select the edge $\langle v_{i',j'+1}, v_{i,j} \rangle$ first.

We emulate uncertainty by pausing 10% randomly chosen agents every k timesteps. Each pause lasts for k timesteps. If an agent is executing a *wait* action when paused, it will stay at its current node for $k+1$ timesteps; if an agent is executing a *move* action when paused, it will arrive at the adjacent node after $k+1$ timesteps. For fair comparison, we pause the same subset of agents at the same timestep for all the algorithms. We measure the sum-of-costs, i.e., the total timesteps taken by all agents to arrive at their goal locations. Given a k setting, we run the path execution for 10 times with different random subsets of agents paused. Overall, there are 250 experiment setups for each map (25 agent placements constructed from senarios files $\times$ 10 uncertainty emulations). We present the average results for all the setups along with the 95% confidence interval. We implement the algorithms in C++ and run the experiments on a machine with Intel Core i9-9820X 3.30GHz CPU and 64GB memory.

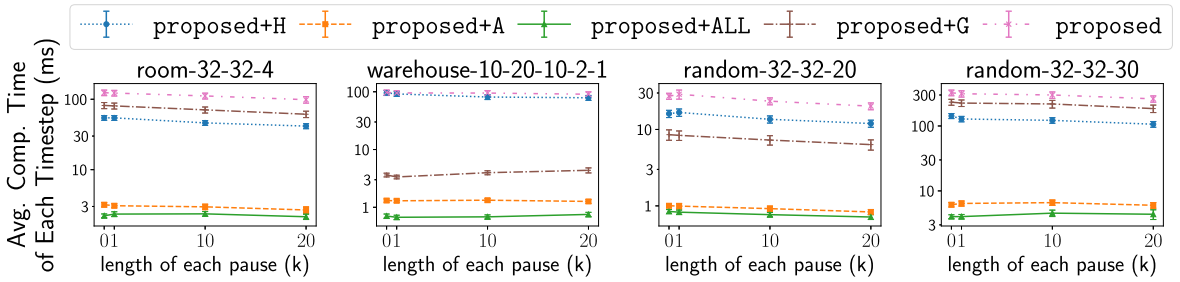


Fig. 8. Comparison of different versions of feasibility test for 40 agents.

6.1. Compare different versions of feasibility test

First, we test different versions of feasibility test in our proposed framework using the heuristic approach for coordinating path execution: (1) `proposed` is the original feasibility test (Algorithm 1) without any optimization. It uses the breadth-first search to detect cycles in the LDG, and does not group the agents; (2) `proposed+H` is the two-level feasibility test which incorporates the heuristic feasibility test (Algorithm 2) to accelerate the original feasibility test; (3) `proposed+A` implements the connection array to optimize cycle detection in the LDG; (4) `proposed+G` groups agents to reduce the number of unsettled edge pairs; (5) `proposed+ALL` combines all optimization methods from (2)-(4).

Fig. 8 compares the average computational time per timestep among different versions of feasibility test for 40 agents. Since all versions yield the same sum-of-costs, we omit the sum-of-costs comparison in this section. It is evident that `proposed+A` significantly improves the performance, followed by `proposed+H` and `proposed+G`. After applying all optimizations, the average computational time per timestep is reduced by up to 99% (the vertical axis is in log scale) compared with the original feasibility test without any optimization.

6.2. Compare different approaches in coordinating path execution

Second, we compare different approaches for selecting agents to unblock when coordinating path execution in our proposed framework: the naive, semi-naive and heuristic approaches. In these experiments, we use the `proposed+ALL` feasibility test. Fig. 9 shows the results for 30 agents. In the experiments, we set a time limit of 6 minutes for each run. The naive approach does not manage to finish within the time limit for a small portion of 250 experiment setups. Thus, we present the average result for only those runs in which the naive approach is completed. The number of agents is limited to 30 because the naive approach runs too slow for more agents.

As seen from the upper row of Fig. 9, the heuristic approach produces almost the same sum-of-costs as the other two approaches. But the three approaches differ significantly in their computational times. The lower row of Fig. 9 compares the average computational time per timestep in path execution. The naive and semi-naive approaches take significantly higher computational time than the heuristic approach (the vertical axis is in log scale). This demonstrates the computational efficiency of the heuristic approach in identifying a good set of agents to unblock.

6.3. Compare with existing methods

Finally, we compare the heuristic approach for coordinating path execution with five existing methods.

- The first method is a baseline method that follows precomputed precedence dependencies among agents by applying the MCP execution policy [9]. In our experiments, it follows the acyclic complete LDG derived from the EECBS solution as described earlier.
- The second method is a replanning method that replans the paths of agents whenever any agent not yet reaching its goal location is delayed (by the pause described earlier). We again run EECBS with the suboptimality factor set to 1.1 for replanning. For fair comparison with our method, replanning does not know the length of the pause and it assumes that the pause will finish in the current timestep. Thus, if the pause lasts for multiple timesteps, replanning is carried out repeatedly. Since EECBS cannot always be completed successfully in a predefined timeout (60 s) especially as the number of agents increases, when the replan fails, we use the path returned by the most recent successful replan to coordinate the agents with the baseline method for the current timestep. The computation time for unsuccessful replans is excluded from the experimental results.
- The third method is Causal-PIBT+ [22], which is a state-of-the-art online progressive planning method looking one step ahead to avoid conflicts and guarantee reachability (all agents will reach their goal locations but not necessarily be there at the same time). Causal-PIBT+ uses an offline plan as hints to compensate for short-sightedness. We use the aforesaid EECBS solution as hints for Causal-PIBT+.
- The fourth method is BTPG-optimized [19]. BTPG identifies the precedence dependencies (unsettled edge pairs) that can be switched in any manner without introducing cycles in the dependency graph (LDG). That is, if there are n edge pairs identified, up to

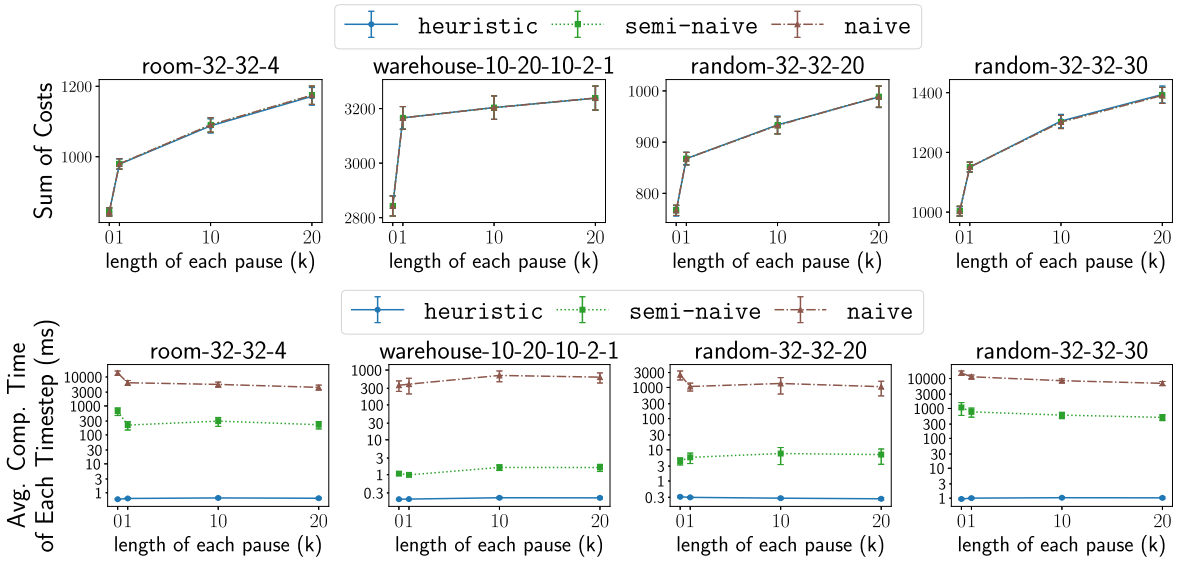


Fig. 9. Comparison of naive, semi-naive and heuristic approaches for 30 agents.

2^n possible dependency graphs (complete LDGs) are required to be acyclic. The advantage is low overhead to switch dependencies during plan execution as there is no need to check for cycles. But this approach may significantly restrict the set of switchable dependencies. In contrast, all dependencies can possibly be switched in our method (as long as one acyclic complete LDG is identified, even if switching dependencies in other ways create cycles).

- The fifth method is GSES (Graph-based Switchable Edge Search) [16]. SES (Switchable Edge Search) defines a switchable dependency graph that allows for the modification of some precedence relations. It uses an A*-style algorithm to find a new dependency graph based on the switchable graph that minimizes the sum-of-costs of the agents when a delay is detected. Graph-based SES leverages the graph properties of a dependency graph to improve the efficiency of SES. In our experiments, GSES takes the plan without following conflicts converted from the EECBS solution (described earlier) as the initial input.

Our proposed algorithm guarantees a 100% success rate across all 250 experiment setups in every map. However, other methods (BTPG, Causal-PIBT+, and replanning) may fail due to various issues. For instance, the BTPG code released by the authors encounters segmentation faults in certain setups, and the Causal-PIBT+ code released by the authors does not always find a solution for all agents to arrive at their goal locations. In such cases, we exclude the failed experiment setups from the results and present the average results for the setups successfully completed by all methods. We also observe that the GSES code released by the authors fails to prevent all conflicts among agents during path execution, e.g., two agents may occupy the same location at the same timestep. Such conflicts are detected in the simulation of both the initial dependency graph and the replanned dependency graphs. Attempts to execute the replanned dependency graphs in our own simulator also fail due to the presence of deadlocks in these graphs. This issue is critical because it can result in the sum-of-costs significantly smaller than the theoretical optimal value, as evident from the experimental results.

Fig. 10 shows the results for 30 agents that compare our proposed method with all optimizations (the heuristic approach for coordinating path execution with the `proposed+ALL` feasibility test) and the five existing methods. The number of agents is limited to 30 because GSES runs too slow for more agents. As seen from the upper row of Fig. 10, when $k = 0$ (no pause in path execution), the methods except Causal-PIBT+ and GSES have similar sum-of-costs. When $k > 0$, our method substantially reduces the sum-of-costs compared to the baseline method. The improvement generally increases with the length of each pause. The replanning method often produces lower sum-of-costs than our method. This is because replanning can also change the paths for agents to travel, while our method keeps the paths of agents unchanged. It can be seen that the sum-of-costs difference between our method and replanning is much smaller than that between our method and the baseline method. This implies that simply adjusting the order for agents to occupy common nodes can attain most of the benefits by replanning in optimization. The sum-of-costs of BTPG is usually between the baseline method and our method. This is because our method allows more switchable dependencies than BTPG. In addition, BTPG does not perform well in the last map random-32-32-30 where the ratio of obstacles is increased, probably because few dependencies are switchable by using BTPG in this case. The sum-of-costs of GSES is significantly smaller than all other methods even when $k = 0$. Given that GSES only searches for a new dependency graph when a delay occurs, the sum-of-costs of GSES should be the same as those of the baseline and replanning methods when there is no pause in path execution ($k = 0$), since we feed the same EECBS solution into all methods initially. Unfortunately, after adding a conflict detection block in the GSES code of path execution, a lot of conflicts among agents are detected. This shows that the GSES implementation released by the authors is not correct and fails to avoid all conflicts. Causal-PIBT+ performs worse than the baseline method in many cases, especially when there are more potential conflicts

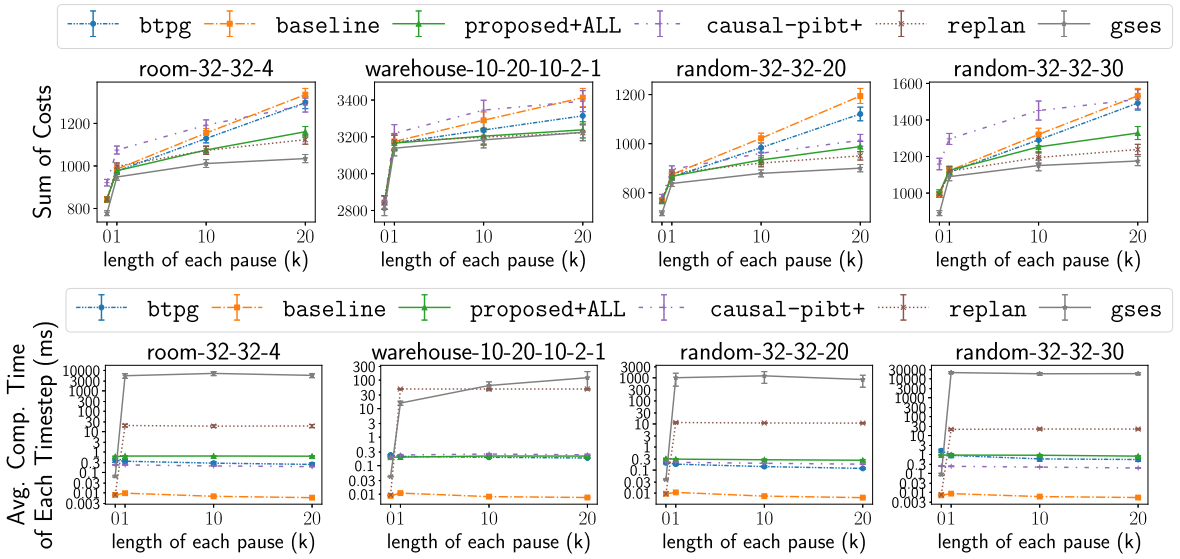


Fig. 10. Comparison of proposed and other methods for 30 agents.

among the agents (in the room map with narrow corridors and the grid map with more obstacles). Causal-PIBT+ is less successful in optimizing the sum-of-costs because it looks only a single step ahead in path planning.

The lower row of Fig. 10 compares the average computational time per timestep in path execution. Note that only successful replans are considered in deriving the results for the replanning method. The baseline method maintains fairly low computational times because it relies on a pre-computed dependency graph and requires no additional computation in plan execution. BTPG, Causal-PIBT+, and our proposed method generally exhibit similar computational times when dealing with 40 agents. The replanning method takes much higher computational time than our method. For our method, we observe that running Algorithm 6 on average requires less than 2 feasibility tests and running a feasibility test is much faster than replanning. Although GSES was shown to be faster than replanning with k-Robust CBS in [16], it is much slower than the replanning method with EECBS in our results for most of the maps. We notice that our maps such as the grid maps are more congested (have more obstacles) than those tested in [16], and our delays (pauses) are more frequent than those tested in [16]. This implies that GSES is not competitive in computational efficiency compared to replanning with EECBS in complex situations.

Fig. 11 shows the results of different methods as a function of the number of agents, where k is set to 20. GSES is not included here because it runs too slow for more than 30 agents. The upper row of Fig. 11 shows the sum-of-costs normalized by the number of agents. As can be seen, the relative performance among different methods discussed earlier is maintained over different numbers of agents. The lower row of Fig. 11 compares the average computational time per timestep among various methods. Similar to Fig. 10, only successful replans are included in the results for the replanning method. The computational times for the baseline and Causal-PIBT+ methods remain relatively constant for different numbers of agents. The computational times for BTPG and our method increase at similar rates as the number of agents rises. Notably, the replanning method exhibits significantly higher computational times than our proposed method for up to 80 agents.

In summary, the experimental evaluation shows a trade-off between path execution effectiveness and computational overhead for various execution methods. The baseline method strictly follows a precomputed dependency graph, so it has the lowest execution effectiveness and the lowest computational overhead at runtime. BTPG records some switchable dependencies in a precomputed dependency graph and dynamically choose them in the execution. It has better execution effectiveness and higher computation overhead at runtime than the baseline method, but the dependency switching options permitted by BTPG are limited. Our proposed solution enables much wider dependency switching options than BTPG by actively invoking feasibility tests for the remaining path plan during the execution. It considerably improves execution effectiveness over BTPG with moderate computational overhead. The replanning method further allows the agent paths to change in the execution and achieves even higher execution effectiveness. However, the computational overhead of replanning is also much higher than other methods. The replanning method can be chosen if unexpected delays are rare, while our proposed solution better suits situations where unexpected delays are more common. Furthermore, our proposed solution can be applied to any agent paths predefined by applications such as security patrolling, but the replanning method may not be workable in this case. BTPG can be considered when the computing capability during the execution is highly limited.

We note that our proposed solution needs to repeatedly evaluate path plan feasibility, which is an NP-complete problem as proved in Section 4.2. The scalability of our solution can be affected by the complexity of maps (topology, obstacle density, etc), and the frequency and magnitude of unexpected delays. An important direction of future work is to further improve the computational efficiency of our solution (particularly feasibility tests). One possible approach is to incorporate window-based techniques [20,21] to reduce the scale of feasibility tests. Another possible approach is to leverage parallel and distributed computation to speed up the solution.

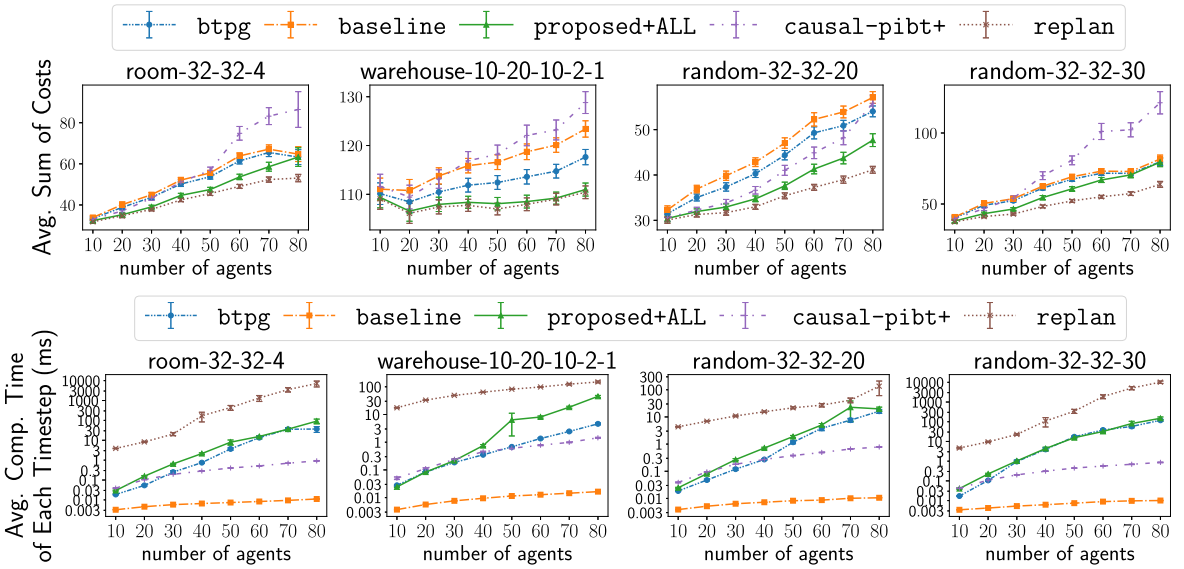


Fig. 11. Comparison of proposed and other methods for different numbers of agents with $k = 20$.

7. Conclusion

We have developed a generic framework for coping with arbitrary delays in the execution of a multi-agent path plan. The framework takes planned paths of agents as input and coordinates agent movements in an online fashion according to the evolving status to avoid conflicts and deadlocks. Empirical evaluations demonstrate the advantages of our algorithms compared with several other methods.

CRediT authorship contribution statement

Yihao Liu: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Xueyan Tang:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis; **Wentong Cai:** Writing – review & editing, Supervision, Project administration, Funding acquisition; **Jingning Li:** Supervision, Resources, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is supported by the Ministry of Education, Singapore, under its Academic Research Fund Tier 2 (Award MOE-T2EP20125-0001) and by the Singtel Cognitive and Artificial Intelligence Lab for Enterprises (SCALE@NTU). The authors would like to thank anonymous reviewers for their helpful suggestions to improve the presentation of this paper.

Data availability

Data will be made available on request.

References

[1] G. Fraser, G. Steinbauer, F. Wotawa, Plan execution in dynamic environments, in: M. Ali, F. Esposito (Eds.), Innovations in Applied Artificial Intelligence, Springer Berlin Heidelberg, 2005, pp. 208–217.
[2] H. Ma, W. Hönig, L. Cohen, T. Uras, H. Xu, T.K.S. Kumar, N. Ayanian, S. Koenig, Overview: a hierarchical framework for plan generation and execution in multirobot systems, IEEE Intell. Syst. 32 (6) (2017) 6–12.
[3] Y. Liu, X. Tang, W. Cai, J. Li, Multi-agent path execution with uncertainty, in: Proceedings of the 17th International Symposium on Combinatorial Search (SoCS), 2024, pp. 64–72.
[4] R. Stern, N.R. Sturtevant, A. Felner, S. Koenig, H. Ma, T.T. Walker, J. Li, D. Atzmon, L. Cohen, T.K.S. Kumar, E. Boyarski, R. Bartak, Multi-agent pathfinding: definitions, variants, and benchmarks, in: Proceedings of the 12th Annual Symposium on Combinatorial Search (SoCS), 2019, pp. 151–158.

[5] D. Atzmon, R. Stern, A. Felner, G. Wagner, R. Barták, N.-F. Zhou, Robust multi-agent path finding and executing, J. Artif. Intell. Res. 67 (2020) 549–579.
[6] T. Shahar, S. Shekhar, D. Atzmon, A. Saffidine, B. Juba, R. Stern, Safe multi-agent pathfinding with time uncertainty, J. Artif. Intell. Res. 70 (2021) 923–954.
[7] G. Wagner, H. Choset, Path planning for multiple agents under uncertainty, in: Proceedings of the 27th International Conference on Automated Planning and Scheduling (ICAPS), 2017, pp. 577–585.
[8] D. Atzmon, R. Stern, A. Felner, N.R. Sturtevant, S. Koenig, Probabilistic robust multi-agent path finding, in: Proceedings of the 30th International Conference on Automated Planning and Scheduling (ICAPS), 2020, pp. 29–37.
[9] H. Ma, T.S. Kumar, S. Koenig, Multi-agent path finding with delay probabilities, in: Proceedings of the 31st AAAI Conference on Artificial Intelligence, 2017, pp. 3605–3612.
[10] K. Okumura, F. Bonnet, Y. Tamura, X. Défago, Offline time-independent multiagent path planning, IEEE Trans. Rob. 39 (4) (2023) 2720–2737.
[11] W. Hönig, T.K.S. Kumar, L. Cohen, H. Ma, H. Xu, N. Ayanian, S. Koenig, Multi-agent path finding with kinematic constraints, in: Proceedings of the 26th International Conference on Automated Planning and Scheduling (ICAPS), 2016, pp. 477–485.
[12] W. Hönig, S. Keisel, A. Tinka, J.W. Durham, N. Ayanian, Persistent and robust execution of MAPF schedules in warehouses, IEEE Rob. Autom. Lett. 4 (2) (2019) 1125–1131.
[13] A. Berndt, N.V. Duijkeren, L. Palmieri, T. Keviczky, A feedback scheme to reorder a multi-agent execution schedule by persistently optimizing a switchable action dependency graph, (2020). arXiv preprint arXiv:2010.05254
[14] A. Coskun, J. O'Kane, M. Valtorta, Deadlock-free online plan repair in multi-robot coordination with disturbances, in: Proceedings of the International FLAIRS Conference, 34, 2021.
[15] A. Paul, Y. Feng, J. Li, A fast rescheduling algorithm for real-time multi-robot coordination [extended abstract], in: Proceedings of the 16th International Symposium on Combinatorial Search (SoCS), 2023, pp. 175–176.
[16] Y. Feng, A. Paul, Z. Chen, J. Li, A real-time rescheduling algorithm for multi-robot plan execution, in: Proceedings of the 34th International Conference on Automated Planning and Scheduling (ICAPS), 2024, pp. 201–209. https://github.com/YinggggFeng/Switchable-Edge-Search.
[17] J. Švancara, E. Tignon, R. Barták, T. Schaub, P. Wanko, R. Kaminski, Multi-agent pathfinding with predefined paths: to wait, or not to wait, that is the question [extended abstract], in: Proceedings of the 16th International Symposium on Combinatorial Search (SoCS), 2023, pp. 185–186.
[18] J. Kottinger, T. Geft, S. Almagor, O. Salzman, M. Lahijanian, Introducing delays in multi agent path finding, in: Proceedings of the 17th International Symposium on Combinatorial Search (SoCS), 2024, pp. 37–45.
[19] Y. Su, R. Veerapaneni, J. Li, Bidirectional temporal plan graph: enabling switchable passing orders for more efficient multi-agent path finding plan execution, in: Proceedings of the 38th AAAI Conference on Artificial Intelligence, 2024, pp. 17559–17566. https://github.com/YifanSu1301/BTPG.
[20] Y. Zhang, Z. Chen, D. Harabor, P.L. Bodic, P.J. Stuckey, Planning and execution in multi-agent path finding: models and algorithms, in: Proceedings of the 34th International Conference on Automated Planning and Scheduling (ICAPS), 2024, pp. 707–715.
[21] Y. Zhang, Z. Chen, D. Harabor, P.L. Bodic, P.J. Stuckey, Concurrent planning and execution in lifelong multi-agent path finding with delay probabilities, in: Proceedings of the 39th AAAI Conference on Artificial Intelligence, 2025, pp. 23387–23394.
[22] K. Okumura, Y. Tamura, X. Défago, Time-independent planning for multiple moving agents, in: Proceedings of the 35th AAAI Conference on Artificial Intelligence, 2021, pp. 11299–11307. https://github.com/Kei18/time-independent-planning.
[23] A. Mascis, D. Pacciarelli, Job-shop scheduling with blocking and no-wait constraints, Eur. J. Oper. Res. 143 (3) (2002) 498–517.
[24] R.M. Karp, Reducibility among combinatorial problems, in: R.E. Miller, J.W. Thatcher, J.D. Bohlinger (Eds.), Complexity of Computer Computations, Springer US, Boston, MA, 1972, pp. 85–103.
[25] E.M. Arkin, A. Banik, P. Carmi, G. Citovsky, M.J. Katz, J.S. Mitchell, M. Simakov, Selecting and covering colored points, Discrete Appl. Math. 250 (2018) 75–86.
[26] C. Arbib, G.F. Italiano, A. Panconesi, Predicting deadlock in store-and-forward networks, Networks 20 (1990) 861–881.
[27] R. Tarjan, Depth-first search and linear graph algorithms, SIAM J. Comput. 1 (2) (1972) 146–160.
[28] J. Li, W. Ruml, S. Koenig, EECBS: a bounded-suboptimal search for multi-agent path finding, in: Proceedings of the 35th AAAI Conference on Artificial Intelligence, 2021, pp. 12353–12362. https://github.com/Jiaoyang-Li/EECBS.
[29] G. Sharon, R. Stern, A. Felner, N.R. Sturtevant, Conflict-based search for optimal multi-agent pathfinding, Artif. Intell. 219 (2015) 40–66.